

**Author: Leon Calvin Long II**

**Contact: [leonlong.research@gmail.com](mailto:leonlong.research@gmail.com)**

## **Semantic Tokenization**

*A Framework for Meaning-Preserving Identity Resolution*

A Companion Paper to *Canonical Tokenization*

Document Version:

1.0

Publication Date:

July 2025

Document Series:

Token Governance Architecture Series — Volume II

Companion Document:

Canonical Tokenization: A Framework for Structural Identity Normalization (Volume I)

Domain:

Enterprise Architecture / Identity Management / Data Governance

Status:

Final — Version 1.0

## Table of Contents

---

### **Abstract**

### **1. Introduction**

### **2. Foundational Concepts**

2.1 Definition of a Semantic Token

2.2 Semantic vs. Canonical Tokenization

2.3 Token Meaning as a First-Class Attribute

### **3. System Architecture**

3.1 Layered Architecture Overview

3.2 Semantic Token Schema

3.3 Semantic Resolution Engine

3.4 Integration with Canonical Tokenization

### **4. Semantic Fields**

4.1 Definition and Purpose

4.2 Semantic Field Registry

4.3 Field Binding

4.4 Cross-Field Token Behavior

4.5 Field Versioning

### **5. Semantic Normalization**

5.1 Purpose of Semantic Normalization

5.2 Semantic Normal Form (SNF)

5.3 Normalization Pipeline

5.4 Normalization Rules Engine

5.5 Handling Normalization Failures

### **6. Semantic Provenance**

6.1 Definition and Importance

6.2 Provenance Chain Structure

6.3 Provenance Event Types

6.4 Provenance Authority

6.5 Provenance Querying and Audit

6.6 Provenance and Trust Scoring

## **7. Semantic Lifecycle**

7.1 Overview of the Semantic Lifecycle Model

7.2 Lifecycle States

7.3 Lifecycle Transitions

7.4 Lifecycle Policies

7.5 Lifecycle and Downstream Systems

## **8. Semantic Equivalence**

8.1 Definition of Semantic Equivalence

8.2 Equivalence Classes

8.3 Types of Semantic Equivalence

8.4 Equivalence Resolution Algorithm

8.5 Equivalence Disputes

8.6 Equivalence Maintenance

## **9. Semantic Triggers**

9.1 Definition and Purpose

9.2 Trigger Architecture

9.3 Trigger Condition Language

9.4 Trigger Action Types

9.5 Trigger Governance

9.6 Example Trigger Scenarios

## **10. Semantic Compliance**

10.1 Compliance as a Semantic Concern

10.2 Compliance Tag Schema

10.3 Compliance Frameworks Supported

10.4 Semantic Compliance Evaluation

10.5 Compliance Violations and Remediation

## 10.6 Compliance Reporting

## 11. Semantic Interoperability

### 11.1 The Interoperability Problem

### 11.2 Interoperability Mapping Architecture

### 11.3 Mapping Types

### 11.4 Semantic Translation Protocol

### 11.5 Interoperability with External Ontologies

### 11.6 Cross-Organization Semantic Contracts

### 11.7 Federation and Distributed Semantic Stores

## 12. Reductions to Practice

### 12.1 Implementation Overview

### 12.2 Reduction 1: Enterprise Customer Identity Resolution

### 12.3 Reduction 2: Regulatory Reporting Semantic Compliance

### 12.4 Reduction 3: Cross-Organization Data Exchange

### 12.5 Reduction 4: AI/ML Training Data Governance

## 13. Discussion

## 14. Conclusion

## Glossary

### Abstract

This paper introduces **Semantic Tokenization**, a formal framework for the assignment, management, and governance of meaning-bearing identifiers within enterprise data architectures. Whereas canonical tokenization — as described in the companion document *Canonical Tokenization: A Framework for Structural Identity Normalization* — addresses the normalization of syntactic and structural variation in token representation, semantic tokenization addresses an orthogonal and complementary problem: the normalization of interpretive ambiguity. A canonical token resolves the question of *what form* an identifier takes; a semantic token resolves the question of *what it means* within a defined domain of discourse.

The central thesis of this paper is that syntactic equivalence is a necessary but insufficient condition for semantic interoperability. Two tokens that are canonically identical may carry materially different meanings across organizational contexts, temporal windows, and domain boundaries. Without a principled mechanism for encoding, versioning, and governing token meaning, any system that relies on token-based identity is subject to silent misinterpretation, compliance failure, and irreconcilable data conflicts.

This paper defines the **Semantic Token** as a five-tuple  $ST = (CT, SF, SN, SP, SL)$ , comprising a canonical token, a semantic field, a semantic normal form, a semantic provenance chain, and a semantic lifecycle state. The framework specifies eight core components: (1) **Semantic Fields**, which provide bounded domains of meaning; (2) **Semantic Normalization**, which resolves interpretive variation to a canonical meaning representation; (3) **Semantic Provenance**, which records the complete lineage of a token's meaning; (4) **Semantic Lifecycle**, which governs the operational state of every token over time; (5) **Semantic Equivalence**, which formalizes when two or more tokens refer to the same underlying concept; (6) **Semantic Triggers**, which automate responses to semantic state changes; (7) **Semantic Compliance**, which ensures tokens satisfy applicable regulatory and governance obligations; and (8) **Semantic Interoperability**, which enables meaning-preserving token exchange across system and organizational boundaries.

The paper concludes with four concrete reductions to practice covering enterprise customer identity resolution, regulatory reporting compliance, cross-organization data exchange, and AI/ML training data governance. Collectively, these contributions establish semantic tokenization as an enterprise-grade standard for meaning-preserving identity management, suitable for deployment in regulated industries, federated data architectures, and any environment in which the correct interpretation of an identifier carries material consequence.

## 1. Introduction

---

The proliferation of data-intensive enterprise architectures has produced a structural dependency on token-based identity: the practice of representing entities, concepts, attributes, and relationships through discrete identifiers that can be stored, transmitted, and resolved across system boundaries. The companion paper to this document, *Canonical Tokenization: A Framework for Structural Identity Normalization*, establishes the foundational layer of this token governance stack — a systematic approach to eliminating syntactic and structural variation in how tokens are formed, represented, and compared. Canonical tokenization is a necessary and significant contribution. It is not, however, sufficient.

The insufficiency of canonical tokenization alone becomes apparent when one recognizes that meaning is not an intrinsic property of form. A token that is structurally normalized — syntactically canonical — may nonetheless carry different meanings in different contexts, different domains, or at different points in time. The word "account," for example, means one thing in the context of a core banking system, another in the context of a customer relationship management platform, and yet another in the context of a human resources information system. If all three systems produce canonically identical tokens for "account," a consuming system that conflates them will silently misinterpret data with potentially severe operational, regulatory, and financial consequences.

This phenomenon — the divergence of meaning from syntactic form — is referred to in this paper as the **Semantic Gap**. The semantic gap is not a data quality problem in the conventional sense; it cannot be resolved by data cleansing, deduplication, or format standardization alone. It is an architectural problem, requiring an architectural solution. That solution is **Semantic Tokenization**: a formal framework that elevates meaning to a first-class attribute of every token, alongside its canonical form, and provides the mechanisms to assign, govern, version, and exchange meaning in a disciplined, auditable, and interoperable manner.

Semantic tokenization is designed as a natural extension of canonical tokenization, not a replacement for it. The two frameworks are architecturally layered: canonical tokenization is a prerequisite that must be satisfied before semantic tokenization can operate. The canonical tokenization layer resolves what form an identifier takes; the semantic tokenization layer resolves what that identifier means within a defined domain. Together they constitute a complete token governance stack: form is resolved at Layer 1, meaning at Layer 2, equivalence and trigger processing at Layer 3, and governance, compliance, and interoperability at Layer 4.

The scope of this paper encompasses the full semantic tokenization lifecycle, from the formal definition of a semantic token through its operational deployment in enterprise environments. The paper is organized to mirror the layered complexity of the subject matter. Section 2 establishes foundational concepts and formal definitions. Section 3 describes the system architecture. Sections 4 through 11 address each of the eight core framework components in depth. Section 12 presents four reductions to practice. Sections 13 and 14 offer discussion and conclusions. A comprehensive glossary appears at the end of the document.

Throughout this paper, cross-references to canonical tokenization concepts are made explicit. Readers are encouraged to treat the two documents as a unified technical specification, with canonical tokenization providing the structural substrate upon which semantic tokenization operates. Where the companion paper speaks to the syntax of identity, this paper speaks to its semantics. The union of both perspectives is, this paper argues, the minimum necessary condition for a trustworthy, interoperable, and governable enterprise data architecture.

## 2. Foundational Concepts

---

### 2.1 Definition of a Semantic Token

The concept of a token, in its most general form, denotes a discrete symbolic unit that stands in for — or refers to — some entity, concept, or value within a system. Tokens may be classified along several dimensions: **Lexical Tokens** are the raw character sequences produced by a lexical analysis pass, prior to any normalization or interpretation; **Syntactic Tokens** are tokens that have been classified and structured according to a formal grammar; and **Canonical Tokens**, as defined in the companion paper, are tokens that have been normalized to a single authoritative structural form, eliminating representational variation. Each of these token types operates at a different level of abstraction, and each is a prerequisite for the next.

A **Semantic Token** is a token that has been elevated to carry not only a canonical identifier but also a bounded, versioned, and governed meaning context. Where a canonical token asserts identity of form, a semantic token asserts identity of meaning. A semantic token is not merely a richer data structure; it is a governed artifact whose meaning has been explicitly assigned by an authoritative process, recorded in a tamper-evident provenance chain, bound to a specific semantic field and field version, and placed in a defined operational state by the semantic lifecycle model.

### Formal Definition — Semantic Token

A **Semantic Token** is defined as the five-tuple:

$$ST = (CT, SF, SN, SP, SL)$$

where:

- **CT** — Canonical Token: the structurally normalized identifier produced by the canonical tokenization layer (per the companion paper).
- **SF** — Semantic Field: the bounded domain of meaning within which this token's interpretation is defined.
- **SN** — Semantic Normal Form: the unique, authoritative meaning representation of this token within its semantic field and field version.
- **SP** — Semantic Provenance: the append-only chain of provenance events recording the complete lineage of this token's meaning.
- **SL** — Semantic Lifecycle State: the current operational state of this token within the semantic lifecycle model.

Each component of this five-tuple is a governed attribute in its own right. The **CT** component is inherited from the canonical tokenization layer and is immutable once assigned. The **SF** component is established at field binding time and is versioned. The **SN** component is computed by the normalization pipeline and is updated as field definitions evolve. The **SP** component is an append-only log that grows over the lifetime of the token. The **SL** component transitions according to the lifecycle state machine defined in Section 7. All five components are jointly necessary; a token that lacks any one of them does not qualify as a semantic token in the sense defined by this framework.



The distinction between semantic tokens and the token types that precede them in the processing hierarchy is substantive, not merely terminological. A lexical token carries no normalization and no meaning; it is a raw textual artifact. A syntactic token carries grammatical classification but no semantic interpretation. A canonical token carries normalized form but treats meaning as incidental or external. A semantic token treats meaning as intrinsic, versioned, and governed — a property of the token itself, not of the context in which it happens to be interpreted. This shift — from meaning as context-dependent inference to meaning as explicitly encoded artifact — is the defining contribution of the semantic tokenization framework.

## 2.2 Semantic vs. Canonical Tokenization

**Canonical Tokenization**, as described in the companion paper, addresses the problem of *structural ambiguity*: the condition in which the same entity may be referred to by multiple syntactically distinct representations. "IBM," "I.B.M.," and "International Business Machines Corporation" are, under canonical tokenization, resolved to a single canonical form. The canonical tokenization process applies a defined normalization pipeline — including character normalization, case folding, punctuation handling, abbreviation expansion, and format standardization — to produce a token that is unique, stable, and structurally unambiguous. This is a necessary and powerful capability. However, canonical tokenization is, by design, agnostic to meaning. Its task is to ensure that representations that should be the same are treated as the same, regardless of surface variation. It does not concern itself with what the resulting token means.

**Semantic Tokenization** addresses the problem of *interpretive ambiguity*: the condition in which the same canonical token may mean different things in different contexts. Even after canonical normalization has been applied, a token such as "CUSTOMER:00124891" may refer to a retail customer, a wholesale customer, an internal business unit acting as a customer, or a historical record of a now-defunct entity — depending entirely on the organizational context, the data source, the applicable ontology, and the point in time at which the token is interpreted. Semantic tokenization resolves this ambiguity by making the interpretation explicit, authoritative, and part of the token's own structure.

The complementarity of the two frameworks is architectural as well as conceptual. Canonical tokenization is a necessary prerequisite for semantic tokenization: before meaning can be assigned to a token, the token must itself be in a stable, normalized form. One cannot build a provenance chain for a token whose identity is structurally ambiguous. One cannot assign a semantic field to a token that may resolve differently on different systems. Canonical tokenization creates the stable foundation upon which semantic tokenization constructs meaning. In the layered architecture described in Section 3.1, canonical tokenization occupies Layer 1 and semantic tokenization occupies Layer 2; the output of Layer 1 is the mandatory input to Layer 2.

| Dimension                   | Canonical Tokenization              | Semantic Tokenization                                |
|-----------------------------|-------------------------------------|------------------------------------------------------|
| <b>Problem Addressed</b>    | Structural / syntactic ambiguity    | Interpretive / semantic ambiguity                    |
| <b>Normalization Target</b> | Token form (representation)         | Token meaning (interpretation)                       |
| <b>Input</b>                | Raw or lexical token                | Canonical token (from Layer 1)                       |
| <b>Output</b>               | Canonical token (CT)                | Semantic token (ST = CT + SF + SN + SP + SL)         |
| <b>Governing Artifact</b>   | Canonical normalization rules       | Semantic field ontology + normalization rules engine |
| <b>Dependency</b>           | None (base layer)                   | Requires canonical tokenization as prerequisite      |
| <b>Provenance</b>           | Structural normalization log        | Full meaning lineage (provenance chain)              |
| <b>Lifecycle</b>            | Implicit (token exists or does not) | Explicit seven-state lifecycle model                 |
| <b>Interoperability</b>     | Structural alignment                | Meaning-preserving cross-system translation          |

## 2.3 Token Meaning as a First-Class Attribute

A central premise of this framework is that token meaning must be treated as a first-class attribute — that is, as an intrinsic, explicitly encoded, and actively governed property of the token itself, rather than as a secondary concern delegated to downstream consuming systems. In the absence of this treatment, meaning is left as an implicit, informal, and ungoverned property that is inferred differently by different systems, interpreted differently by different analysts, and subject to silent mutation over time. The consequences of this approach are well documented in enterprise data management practice: data integration failures, incorrect analytics, regulatory non-compliance, and the costly proliferation of system-specific "data dictionaries" that are perpetually out of date and never fully reconciled.

The elevation of meaning to a first-class attribute has several concrete architectural implications. First, meaning must be *explicitly encoded*: the semantic normal form must be a structured, machine-readable representation of the token's meaning, not a free-text description. Second, meaning must be *versioned*: because meaning can and does change over time as ontologies evolve, regulatory requirements shift, and organizational structures change, every semantic normal form must be associated with the version of the semantic field under which it was produced. Third, meaning must be *governed*: the assignment, modification, and deprecation of meaning must be subject to formal governance processes with defined authorities, approval workflows, and audit records. Fourth, meaning must be *propagated*: when a token's meaning changes, all downstream systems that hold or use that token must be notified, and the change must be recorded in the token's provenance chain.

This approach stands in deliberate contrast to the informal convention of relying on data dictionaries, business glossaries, or organizational knowledge to mediate meaning at the point of use. Such conventions are valuable as human-readable documentation aids but are architecturally insufficient for machine-to-machine interoperability, automated compliance evaluation, and large-scale data governance. The semantic tokenization framework provides the machine-executable counterpart to human-readable documentation: a formal, queryable, versioned representation of meaning that systems can rely on as an authoritative source of semantic truth.

The treatment of meaning as a first-class attribute also has important implications for trust. In regulated environments — financial services, healthcare, government — the correctness of data interpretation is itself a compliance obligation. A system that cannot demonstrate, with audit-ready evidence, how it interpreted a given token at a given point in time is a system that cannot fully satisfy regulatory scrutiny. The semantic provenance chain (Section 6) and compliance tagging mechanism (Section 10) are direct responses to this requirement. They transform meaning from an ephemeral, context-dependent inference into a durable, tamper-evident record — one that can be queried, audited, and presented as evidence of interpretive correctness.

### 3. System Architecture

---

#### 3.1 Layered Architecture Overview

The semantic tokenization framework is organized as a four-layer architecture in which each layer builds upon the layer below it and provides a defined set of services to the layer above. This layered structure ensures separation of concerns, enables independent evolution of each layer, and allows organizations to adopt the framework incrementally — beginning with canonical tokenization and adding semantic enrichment, resolution, and governance capabilities as maturity develops.

| Layer          | Name                       | Responsibility                                                                                                                    | Output Artifact                                         |
|----------------|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Layer 0</b> | Raw Input                  | Ingestion of raw token representations from source systems; no normalization applied                                              | Raw token (lexical form)                                |
| <b>Layer 1</b> | Canonical Tokenization     | Structural normalization per the companion paper; elimination of syntactic variation; production of a stable canonical identifier | Canonical Token (CT)                                    |
| <b>Layer 2</b> | Semantic Tokenization      | Semantic field binding; normalization to Semantic Normal Form; provenance chain initialization; lifecycle state assignment        | Semantic Token (ST)                                     |
| <b>Layer 3</b> | Semantic Resolution Engine | Equivalence evaluation; trigger processing; compliance checking; interoperability mapping                                         | Resolved Semantic Token with full metadata              |
| <b>Layer 4</b> | Semantic Governance        | Lifecycle management; compliance reporting; interoperability contract management; federated coordination                          | Governance artifacts, compliance reports, audit records |

The progression through these layers transforms a raw, uninterpreted textual identifier into a fully governed semantic artifact. At Layer 0, the system receives whatever representation the source provides — potentially inconsistent, ambiguous, or malformed. Layer 1 applies canonical normalization, as specified in the companion paper, producing a stable and unique canonical identifier. Layer 2 is the entry point of the semantic tokenization framework proper; it enriches the canonical token with semantic field binding, computes the semantic normal form, initializes the provenance chain, and assigns the initial lifecycle state. Layer 3 operates upon the semantic token to evaluate equivalence, dispatch triggers, check compliance, and produce interoperability mappings. Layer 4 provides the governance infrastructure that keeps all lower layers functioning correctly over time: managing field definitions, ontologies, equivalence classes, trigger registries, compliance frameworks, and interoperability contracts.

The layered architecture enforces a strict processing order: no layer may operate on inputs that have not been processed by all preceding layers. A semantic token cannot be produced without a canonical token; a resolved semantic token cannot be produced without a semantic token. This ordering is not merely a matter of implementation convenience — it reflects a principled dependency structure in which each layer's correctness depends on the correctness of the layers below it. An organization that attempts to perform semantic resolution on un-normalized canonical tokens will find that its equivalence classes are unreliable, its provenance chains are ambiguous, and its compliance evaluations are incomplete.

### 3.2 Semantic Token Schema

The full semantic token is represented as a structured record with a defined schema. Every field in the schema is typed, has defined cardinality constraints, and serves a specific architectural purpose within the framework. The schema is designed to be serializable to standard enterprise data formats (JSON, XML, Avro, Parquet) and to support efficient indexing, querying, and streaming distribution.

| Field Name           | Type                   | Cardinality         | Purpose                                                                                                                                             |
|----------------------|------------------------|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| token_id             | UUID                   | 1                   | Globally unique identifier for this semantic token; shared namespace with canonical token registry                                                  |
| canonical_id         | UUID                   | 1                   | Reference to the canonical token (CT) produced by Layer 1; foreign key to canonical token registry                                                  |
| semantic_field_id    | String (URI)           | 1..N                | Identifier of the semantic field to which this token is bound; may be multi-valued for cross-field tokens                                           |
| semantic_normal_form | Object (SNF)           | 1 per field binding | The authoritative meaning representation for this token within its semantic field; includes concept URI, preferred label, field-specific attributes |
| provenance_chain     | Array<ProvenanceEvent> | 1..unbounded        | Ordered, append-only log of all provenance events; records complete meaning lineage                                                                 |
| lifecycle_state      | Enum                   | 1                   | Current operational state: DRAFT   PENDING_REVIEW   ACTIVE   DEPRECATED   RETIRED   QUARANTINED   CONTESTED                                         |
| equivalence_class_id | UUID                   | 0..N                | References to equivalence classes of which this token is a member; one per applicable field                                                         |

| Field Name                | Type                  | Cardinality | Purpose                                                                             |
|---------------------------|-----------------------|-------------|-------------------------------------------------------------------------------------|
| trigger_set               | Array<TriggerRef>     | 0..N        | References to active triggers applicable to this token's current state              |
| compliance_tags           | Array<ComplianceTag>  | 0..N        | Set of compliance evaluations for applicable regulatory and governance frameworks   |
| interoperability_mappings | Array<InteropMapping> | 0..N        | Mappings to and from external system tokens; supports semantic translation protocol |
| trust_score               | Decimal (0.0–1.0)     | 1           | Computed trust score based on provenance chain depth and authority quality          |
| created_at                | ISO 8601 Timestamp    | 1           | Timestamp of semantic token creation (distinct from canonical token creation)       |
| last_modified_at          | ISO 8601 Timestamp    | 1           | Timestamp of most recent modification to any mutable field                          |
| schema_version            | String (SemVer)       | 1           | Version of this token schema; enables forward and backward compatibility management |

The schema is designed to be extensible. Organizations may define domain-specific extensions by adding fields under a reserved namespace prefix (e.g., `ext.*`), provided that extension fields are registered with the Semantic Governance Board and do not conflict with the core schema. Core schema fields are immutable once the schema version has been released; modifications to core fields require a schema version increment and a corresponding migration plan for all existing tokens.

### 3.3 Semantic Resolution Engine

The **Semantic Resolution Engine (SRE)** is the central processing component of the semantic tokenization framework. Its responsibility is to accept a canonical token and a resolution context as input, and to produce a fully populated semantic token as output. The SRE is a stateful service that maintains access to all framework registries — the semantic field registry, the equivalence class registry, the trigger registry, the compliance framework registry, and the interoperability mapping registry — and coordinates the operation of seven internal components.

The **Field Classifier** is the first component to operate on an incoming canonical token. Using the resolution context — which includes signals such as the data source identifier, the pipeline context tag, the requesting user's organizational role, and any explicit field override supplied by the caller — the Field Classifier determines which semantic field or fields the token should be bound to. The Field Classifier consults the Semantic Field Registry (Section 4.2) and applies the field binding rules defined in Section 4.3 to produce a ranked list of candidate field bindings. In cases of unresolvable ambiguity, the Field Classifier may emit multiple candidate bindings for downstream human review.

The **Normal Form Generator** receives the candidate field bindings and applies the normalization pipeline described in Section 5.3 to each binding, producing a Semantic Normal Form (SNF) for each. The Normal Form Generator consults the field's governing ontology, applies the Normalization Rules Engine (Section 5.4), and validates the resulting SNF against the field schema. The **Provenance Recorder** operates in parallel with all other components, capturing each processing event as a provenance entry in the token's provenance chain, including the actor, timestamp, action type, input state, output state, and authority reference for each event.

The **Equivalence Evaluator** compares the resolved SNF against the Equivalence Class Registry to determine whether the token is a member of one or more equivalence classes. The **Trigger Dispatcher** evaluates all active triggers in the Trigger Registry against the token's current state and dispatches any applicable trigger actions. The **Compliance Checker** identifies applicable compliance frameworks based on the token's field binding and token type, evaluates each framework's requirements against the token's state, and updates the token's compliance tags. The **Interoperability Mapper** looks up applicable outbound interoperability mappings in the mapping registry and populates the token's `interoperability_mappings` field.



### 3.4 Integration with Canonical Tokenization

The integration between the semantic tokenization framework and the canonical tokenization layer is governed by a defined handoff protocol. When the canonical tokenization layer produces a canonical token, it emits a *CANONICAL\_RESOLVED* event to the semantic enrichment pipeline. This event includes the canonical token's unique identifier (`canonical_id`), its canonical normal form, its source system identifier, and any context metadata that was available at the time of canonical resolution. The semantic enrichment pipeline receives this event and initiates the SRE processing workflow described in Section 3.3.

The relationship between canonical tokens and semantic tokens is one-to-many in general: a single canonical token may give rise to multiple semantic tokens, one for each semantic field to which it is bound. This is not a defect but a feature of the framework's design. The same canonical identifier — say, a canonically normalized product code — may have a distinct and different meaning in a logistics field, a pricing field, and a regulatory reporting field. Each of these meanings deserves its own semantic token, its own provenance chain, and its own lifecycle management. The `canonical_id` field in the semantic token schema provides the stable link back to the canonical token from which all semantic tokens in this family are derived.

The shared `token_id` namespace between the canonical and semantic layers ensures that identifiers are globally unique and do not collide. Canonical tokens are issued identifiers from the lower portion of the namespace; semantic tokens are issued identifiers from the upper portion, or are derived from their canonical parent using a defined namespace partitioning scheme. This arrangement allows any system, given a token identifier, to determine whether it holds a canonical or semantic token and to retrieve the complete token record from the appropriate registry. The namespace partitioning scheme is defined as part of the implementation specification and is subject to governance by the Semantic Governance Board.

## 4. Semantic Fields

---

### 4.1 Definition and Purpose

#### Formal Definition — Semantic Field

A **Semantic Field** is a bounded, formally defined domain of meaning within which a token's interpretation is specified, governed, and versioned. A semantic field constitutes the namespace for meaning: two tokens are comparable in meaning only if they share the same semantic field and field version. Outside its defining field, a token's semantic normal form has no authoritative standing.

The concept of a semantic field is borrowed and formalized from linguistic semantics, where it denotes a set of words related by meaning within a domain (e.g., the semantic field of "color" or "kinship"). In this framework, the concept is adapted to the requirements of enterprise data governance: a semantic field is not an informal grouping of related terms but a formally registered, version-controlled, ontology-governed domain whose boundaries are precisely specified and whose authority is explicitly assigned to a field steward.

Semantic fields serve three essential purposes within the framework. First, they act as *meaning namespaces*: by scoping the interpretation of a token to a specific field, they eliminate the possibility of cross-domain misinterpretation. The term "account" means one thing in the banking field and another in the HR field; because each field has its own governing ontology and normalization rules, a token bound to the banking field will never be inadvertently interpreted using HR field semantics. Second, semantic fields act as *governance units*: field-level governance assigns ownership, stewardship, and approval authority at the level of the domain, allowing governance responsibilities to be distributed across organizational units without losing central coherence. Third, semantic fields act as *versioning scopes*: when the meaning of a concept changes within a domain, the field version is incremented, and all tokens bound to that field and version are either migrated or marked as bound to the prior version, preserving the historical accuracy of the provenance chain.

The cardinality relationship between tokens and semantic fields is many-to-many. A single canonical token may be bound to multiple semantic fields — for example, a customer identifier may be bound to both a "Customer Identity" field and a "Regulatory Reporting" field, with different SNFs in each. Conversely, a single semantic field will typically contain thousands or millions of tokens, all interpreted according to the same governing ontology. This many-to-many relationship is managed through the field binding mechanism described in Section 4.3 and represented in the token schema through the multi-valued `semantic_field_id` field.

## 4.2 Semantic Field Registry

The **Semantic Field Registry** is a centralized, authoritative catalog of all recognized semantic fields within the organization's semantic tokenization deployment. Every semantic field that is used to bind tokens must be registered in this registry before any token can be bound to it. Unregistered fields are not valid binding targets; any attempt to bind a token to an unregistered field is rejected by the Field Classifier with an appropriate error event recorded in the provenance chain.

| Field                             | Type                | Description                                                                                                                    |
|-----------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>field_id</code>             | String (URI)        | Globally unique identifier for this semantic field; URI-formatted to facilitate linkage with external ontologies               |
| <code>field_name</code>           | String              | Human-readable canonical name of the field (e.g., "Customer Identity," "Regulatory Reporting," "Product Classification")       |
| <code>domain</code>               | String              | Organizational or industry domain to which this field belongs (e.g., "Financial Services," "Healthcare," "Human Resources")    |
| <code>governing_ontology</code>   | URI + Version       | Reference to the ontology that governs meaning within this field; includes ontology URI and version string                     |
| <code>field_version</code>        | String (SemVer)     | Current version of this field definition; incremented when field boundaries, governing ontology, or normalization rules change |
| <code>owner</code>                | Principal reference | Organizational unit or role responsible for this field; the field steward who approves token bindings and field changes        |
| <code>approved_token_types</code> | Array<String>       | Set of canonical token types that may be bound to this field; enforces type safety at binding time                             |
| <code>parent_field_id</code>      | URI (optional)      | Reference to parent field in the field hierarchy; supports field inheritance of normalization rules and ontology               |

| Field           | Type                     | Description                                                                          |
|-----------------|--------------------------|--------------------------------------------------------------------------------------|
| child_field_ids | Array<URI>               | References to child fields that inherit from this field                              |
| lifecycle_state | Enum                     | Operational state of the field itself: PROPOSED   ACTIVE   DEPRECATED   RETIRED      |
| effective_from  | ISO 8601 Date            | Date from which this field version is effective                                      |
| effective_until | ISO 8601 Date (optional) | Date until which this field version is effective; null for currently active versions |

The field registry supports a hierarchical field structure through the `parent_field_id` and `child_field_ids` fields. Child fields inherit the normalization rules and governing ontology of their parent field, unless explicitly overridden in the child field's definition. This inheritance mechanism allows organizations to define a general field (e.g., "Identity") with common normalization rules and then specialize it into domain-specific sub-fields (e.g., "Customer Identity," "Employee Identity," "Partner Identity") that extend and refine the parent's rules. Field inheritance is strict: a child field may only narrow its parent's behavior, not contradict it. Any normalization rule in a child field that conflicts with a parent field rule is invalid and is rejected at field registration time.

### 4.3 Field Binding

**Field Binding** is the process by which a canonical token is associated with one or more semantic fields at resolution time. Field binding is one of the most consequential operations in the semantic tokenization lifecycle: it determines which governing ontology will be used to normalize the token, which compliance frameworks will be applied, and which downstream systems are eligible recipients of the token. An incorrect field binding produces a semantically incorrect token regardless of how well the subsequent normalization and provenance steps are executed; garbage in, garbage out at the level of meaning.

The Field Classifier uses a defined set of context signals to determine the appropriate field binding for an incoming canonical token. These signals are evaluated in a defined priority order:

1. **Explicit Override:** If the calling system provides an explicit field identifier, this takes unconditional precedence, subject to the caller having appropriate authority to make field binding assertions.
2. **Data Source Signal:** The source system from which the canonical token originated is mapped to one or more candidate semantic fields in the Source System Field Map, a configuration artifact maintained by the Semantic Governance Board.
3. **Pipeline Context Tag:** The data processing pipeline that produced the canonical token resolution event may carry a pipeline context tag that indicates the relevant semantic domain.
4. **User Role Signal:** The organizational role of the user or system requesting resolution may bias field selection toward fields relevant to that role's domain.
5. **Ontological Inference:** In the absence of stronger signals, the Field Classifier may apply lightweight ontological inference — using the canonical token's type, format, and structural characteristics — to produce a probabilistic field ranking.

When multiple candidate fields are returned by the Field Classifier and no explicit override resolves the ambiguity, the framework applies field precedence rules. These rules are defined per token type in the Field Precedence Configuration and specify, for a given token type and a set of candidate fields, which field takes precedence and under what conditions. If no precedence rule applies, the token is bound to all candidate fields simultaneously — producing one semantic token per field — and a `BINDING_AMBIGUITY` event is recorded in each token's provenance chain, flagging the need for human review to confirm or correct the multi-field binding.

#### 4.4 Cross-Field Token Behavior

Some tokens legitimately span multiple semantic fields. A customer identifier that appears in both a "Customer Identity" field and a "Regulatory Reporting" field is not an ambiguous token — it is a token whose meaning is fully defined in each field, and both meanings are simultaneously relevant and valid. The framework accommodates this reality through the multi-valued `semantic_field_id` field in the token schema: a single semantic token may carry multiple field bindings, each with its own SNF, equivalence class membership, and compliance tags.

**Field Intersection Tokens** are a specific category of cross-field tokens that arise when two or more fields share a conceptual overlap. A "product" token may exist at the intersection of a "Product Classification" field and a "Pricing" field; in this intersection, the token's meaning in each field is distinct but related. Field intersection tokens are handled by producing a separate SNF for each field binding and recording a `FIELD_INTERSECTION` event in the provenance chain. The framework does not attempt to merge intersection SNFs into a single representation; each field's SNF is authoritative within that field and must not be conflated with another field's SNF.

Conflict resolution is required when two fields in which a token is simultaneously bound assert contradictory meanings. This may occur when field definitions overlap in ways that were not anticipated at design time, or when the same canonical token is bound to two fields whose governing ontologies classify the same concept differently. When a conflict is detected — identified by the Equivalence Evaluator when it finds that two SNFs for the same canonical token, in fields that share a parent or domain, are mutually inconsistent — the token is transitioned to the `CONTESTED` lifecycle state, a `CONFLICT_DETECTED` event is recorded in the provenance chain, and the token is escalated to the Semantic Governance Board for resolution. The `CONTESTED` state prevents the token from being used in downstream processing until the conflict is resolved.

## 4.5 Field Versioning

Semantic fields are not static artifacts. As organizational structures evolve, regulatory requirements change, and domain knowledge matures, the definitions, boundaries, governing ontologies, and normalization rules of semantic fields must change with them. The framework mandates that all such changes be captured through a formal field versioning mechanism, ensuring that no change to a field's definition silently alters the meaning of tokens that were bound under a prior version.

When a field version is incremented, all tokens bound to the prior version of the field retain their prior-version binding. They are not automatically migrated to the new field version; migration is an explicit governance action that must be approved by the field steward and recorded in the provenance chain of each affected token. During the migration window, both the prior version and the new version of the field are simultaneously active; the SRE is capable of resolving tokens bound to any active field version. Once migration is complete, the prior field version is transitioned to DEPRECATED and eventually to RETIRED, at which point tokens still bound to the retired version are flagged for urgent remediation.

The impact of field version changes on existing tokens varies in severity. Non-breaking changes — such as the addition of new synonym rules or the expansion of the approved token type list — do not invalidate existing SNFs and do not require token migration; affected tokens are updated in place with a FIELD\_VERSION\_UPDATE event in their provenance chain. Breaking changes — such as modifications to the governing ontology that alter the URI or label of one or more concepts, or changes to field boundaries that move concepts from one field to another — do invalidate existing SNFs and require explicit migration. The field steward is responsible for classifying each field version change as breaking or non-breaking and initiating the appropriate migration workflow.

## 5. Semantic Normalization

---

### 5.1 Purpose of Semantic Normalization

Within the layered architecture of the semantic tokenization framework, normalization operates at two distinct levels. Canonical normalization, as described in the companion paper, removes structural variation: it ensures that "IBM," "I.B.M.," and "International Business Machines" all resolve to the same canonical form. Semantic normalization operates at a higher level of abstraction: it removes *interpretive variation* — the variation that arises not from differences in how a concept is spelled or formatted, but from differences in how it is understood, categorized, and related to other concepts within a given domain of discourse.

The goal of semantic normalization is precise: given any valid expression of a concept within a defined semantic field, produce a single, unambiguous **Semantic Normal Form (SNF)** that is the canonical meaning representation of that concept within that field and field version. Two tokens that produce the same SNF are, by definition, semantically equivalent within the scope of that field. Semantic normalization is therefore the foundation upon which semantic equivalence (Section 8) rests.

The necessity of semantic normalization becomes clear through concrete examples. In a healthcare domain, the concepts "myocardial infarction," "heart attack," "MI," and "acute coronary event" all refer to the same underlying clinical entity. A system that treats these as distinct concepts will produce fragmented analytics, inconsistent clinical records, and potentially dangerous reporting failures. Semantic normalization resolves all valid expressions of this concept to a single SNF — in this case, the SNOMED CT concept identifier for myocardial infarction — ensuring that any downstream system that receives a token bound to the healthcare field and normalized to this SNF can correctly identify the underlying clinical entity regardless of how it was originally expressed in the source data.



## 5.2 Semantic Normal Form (SNF)

### Formal Definition — Semantic Normal Form (SNF)

The **Semantic Normal Form (SNF)** is the unique, authoritative representation of a concept's meaning within a given semantic field and field version. The SNF is a structured object rather than a simple string; it carries sufficient information to identify the concept unambiguously, relate it to its governing ontology, and support downstream processing without reference to the original token expression.

**Key property:** The SNF is unique per (canonical\_token\_id, semantic\_field\_id, field\_version) triple. No two distinct concepts in the same field and field version may share an SNF.

T

The SNF includes the following components:

- **Resolved Concept URI:** The URI of the concept within the field's governing ontology. This is the primary identifier of the concept and is the basis for equivalence comparison.
- **Preferred Label:** The field-approved human-readable label for the concept, as defined in the governing ontology. This label is used in compliance reports, audit artifacts, and human-readable token representations.
- **Field-Specific Attributes:** A set of structured attributes that are defined by the field's schema and carry domain-specific information about the concept (e.g., for a financial instrument concept, attributes might include asset class, settlement currency, and regulatory classification).
- **Disambiguation Qualifiers:** A set of qualifier values that distinguish this concept from closely related concepts within the same field. Qualifiers are used when two concepts share similar labels but have different URIs; they provide the additional context needed to confirm correct identification.
- **SNF Version:** The version of the field under which this SNF was computed. SNFs are version-scoped and must not be treated as valid under a different field version without explicit re-validation.
- **Normalization Confidence Score:** A decimal value between 0.0 and 1.0 indicating the normalization pipeline's confidence in the correctness of this SNF assignment. Scores below a configurable threshold (default: 0.85) trigger QUARANTINE and escalation.

### 5.3 Normalization Pipeline

The normalization pipeline is a six-step sequential process that transforms a canonical token and a set of candidate field bindings into one or more fully populated SNFs. Each step is atomic and produces a defined output that is consumed by the next step. The pipeline is implemented as a stateless, horizontally scalable service within the Semantic Resolution Engine.

| Step | Name                           | Description                                                                                                                                                                                                                       | Output                                                                                                  |
|------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| 1    | Canonical Token Reception      | Receive canonical token from Layer 1; validate that the token is in a valid canonical form and that its <code>canonical_id</code> is registered in the canonical token registry                                                   | Validated canonical token record                                                                        |
| 2    | Candidate Field Identification | Apply the context classifier using all available context signals; produce a ranked list of candidate semantic field bindings                                                                                                      | Ordered list of ( <code>field_id</code> , <code>field_version</code> , <code>confidence</code> ) tuples |
| 3    | Field-Specific Normalization   | For each candidate field binding, apply the field's normalization rules: lexical expansion, synonym resolution, abbreviation expansion, and ontology mapping                                                                      | Candidate concept URI per field binding                                                                 |
| 4    | SNF Resolution                 | Look up the candidate concept URI in the field's governing ontology; retrieve the preferred label, field-specific attributes, and disambiguation qualifiers; compute the normalization confidence score                           | Candidate SNF object per field binding                                                                  |
| 5    | SNF Validation                 | Validate the candidate SNF against the field schema; check that the concept URI exists and is active in the governing ontology version; check that all required field-specific attributes are populated; reject or flag anomalies | Validated SNF or validation error                                                                       |
| 6    | Semantic Token Emission        | Assemble the full semantic token record with the validated SNF attached; initialize the provenance chain with a NORMALIZATION event; assign the initial lifecycle state (DRAFT or PENDING_REVIEW based on field configuration)    | Fully populated semantic token (ST)                                                                     |

## 5.4 Normalization Rules Engine

The **Normalization Rules Engine (NRE)** is the component of the normalization pipeline responsible for Step 3: applying field-specific normalization rules to a canonical token to produce a candidate concept URI. The NRE maintains a versioned rule set for each registered semantic field and applies rules in a defined priority order. Rules are governed artifacts: they are versioned, approved by the field steward, and recorded in the field's rule registry.

The NRE supports four categories of normalization rules:

- **Synonym Rules:** Map one or more synonym expressions to a target concept URI within the field. Synonym rules are the most common rule type and are typically derived from the governing ontology's synonym annotations (e.g., SKOS `skos:altLabel` values).
- **Abbreviation Expansion Rules:** Map abbreviated forms (e.g., "MI" → "myocardial infarction") to their expanded equivalents before ontology lookup. Abbreviation expansion is field-specific; the same abbreviation may expand differently in different fields.
- **Domain-Specific Alias Rules:** Map organizational or system-specific aliases (e.g., legacy system codes, proprietary classification labels) to concept URIs. These rules are particularly important when integrating tokens from legacy systems whose internal vocabularies predate the governing ontology.
- **Cross-Lingual Equivalence Rules:** Map expressions in languages other than the field's primary language to concept URIs, enabling multilingual token normalization within a single field.

Rule precedence is defined as follows: Explicit Override rules (if present) take highest precedence; Abbreviation Expansion rules are applied second; Synonym rules are applied third; Domain-Specific Alias rules are applied fourth; and Cross-Lingual Equivalence rules are applied last. When two rules of equal precedence produce different candidate concept URIs, the conflict is resolved by selecting the rule with the higher confidence weight. If no confidence distinction exists, the conflict is logged and the token is transitioned to QUARANTINED for human review. Rule conflict records are maintained in the field's rule registry and are subject to governance review to prevent persistent conflicts from accumulating.

## 5.5 Handling Normalization Failures

Not every canonical token can be successfully normalized to an SNF on the first attempt. The normalization pipeline recognizes two categories of failure: *unresolvable tokens*, for which no candidate concept URI can be determined, and *partial normalization results*, for which a candidate concept URI is identified but the normalization confidence score falls below the configured threshold.

Unresolvable tokens — those for which the NRE produces no candidate concept URI after exhausting all applicable rule categories — are placed in QUARANTINED lifecycle state with a NORMALIZATION\_FAILURE provenance event. A human review workflow is initiated, presenting the token and all available context information to a designated semantic analyst. The analyst may resolve the failure by: (a) identifying the correct concept URI and recording it as a manual normalization assertion; (b) creating a new normalization rule to handle this class of token; or (c) determining that the token is invalid and should not be normalized, in which case the token is transitioned to RETIRED. All analyst actions are recorded in the provenance chain with the analyst's identity, timestamp, and a rationale statement.

Partial normalization results — those for which an SNF is produced but with a confidence score below the threshold — are emitted as semantic tokens with the `lifecycle_state` set to PENDING\_REVIEW and a PARTIAL\_NORMALIZATION provenance event. The token is usable for read-only purposes (e.g., analytics) but is not eligible for use in compliance-sensitive or mission-critical downstream processes until it has been reviewed and confirmed by a semantic analyst. This approach prioritizes availability over strict correctness for non-critical use cases while preserving the integrity of critical processes.

## 6. Semantic Provenance

---

### 6.1 Definition and Importance

**Semantic Provenance** is the complete, tamper-evident record of a semantic token's meaning lineage: every decision, transformation, assignment, and validation that has contributed to the token's current semantic state, from the moment of its creation to the present. Semantic provenance is the audit trail of meaning. In the same way that data provenance tracks where a data element came from and how it was transformed, semantic provenance tracks how a token's meaning was determined and how that meaning has evolved over the token's lifetime.

The importance of semantic provenance cannot be overstated in environments where the correctness of interpretation carries regulatory, financial, or clinical significance. Consider a financial institution that is required to report certain data elements to a regulatory body in accordance with a mandated reporting ontology. If the regulator challenges the accuracy of a submitted report, the institution must be able to demonstrate — with machine-verifiable, tamper-evident evidence — that every data element in the report was interpreted correctly at the time of submission. Without semantic provenance, this demonstration is impossible. With semantic provenance, the institution can produce a point-in-time snapshot of every token's SNF, the rules that were applied to produce it, the authority that approved it, and the compliance tags that confirmed its regulatory fitness — all in a form that is cryptographically verifiable and legally defensible.

Semantic provenance also serves a critical role in debugging and continuous improvement. When a downstream system reports an unexpected interpretation of a token, the provenance chain provides a complete diagnostic record: which field was bound, which normalization rules were applied, what confidence score was assigned, which equivalence class the token was placed in, and what triggers fired. Without this record, semantic bugs are notoriously difficult to reproduce and diagnose because the incorrect interpretation may have occurred weeks or months before the symptom became visible.

## 6.2 Provenance Chain Structure

The **Provenance Chain** is an ordered, append-only log of provenance events. Once recorded, provenance events may never be modified or deleted; they may only be appended to. This append-only constraint is enforced at the storage layer and is further protected by the cryptographic attestation mechanism described in Section 6.4. The provenance chain is stored as an ordered array of provenance event records within the semantic token; it grows monotonically over the lifetime of the token and is preserved in its entirety even after the token reaches the RETIRED lifecycle state.

| Field                 | Type                                   | Description                                                                                                             |
|-----------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| event_id              | UUID                                   | Globally unique identifier for this provenance event                                                                    |
| timestamp             | ISO 8601 with timezone                 | Precise timestamp of the event, expressed in UTC with sub-second precision                                              |
| actor                 | Principal reference                    | The system principal, user, or governance role that caused or approved this event                                       |
| action_type           | Enum (see 6.3)                         | The category of semantic action represented by this event                                                               |
| input_state           | Object (snapshot)                      | Snapshot of the relevant token state immediately before this event                                                      |
| output_state          | Object (snapshot)                      | Snapshot of the relevant token state immediately after this event                                                       |
| authority_reference   | URI                                    | Reference to the rule, policy, or governance artifact that authorized this event                                        |
| confidence_score      | Decimal (0.0–1.0)                      | Confidence level associated with this event; particularly relevant for normalization and equivalence events             |
| attestation_signature | Base64-encoded cryptographic signature | Cryptographic signature over the event record, produced by the acting authority's private key; enables tamper detection |
| event_notes           | String (optional)                      | Human-readable notes provided by the actor, particularly for HUMAN_REVIEW events                                        |

### 6.3 Provenance Event Types

The framework defines a canonical set of provenance event types. Every event recorded in a provenance chain must be classified as one of these types. The event type determines which fields are required in the event record and which downstream processes are notified of the event. Organizations may extend the event type taxonomy through a governance-approved extension mechanism, but core event types may not be redefined.

| Event Type                  | Trigger Condition                                                    | Key State Change                                                           |
|-----------------------------|----------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>CREATION</b>             | Token is first minted by the semantic tokenization pipeline          | Lifecycle state set to DRAFT; token_id assigned                            |
| <b>FIELD_BINDING</b>        | Token is bound to a semantic field by the Field Classifier           | semantic_field_id populated; field version recorded                        |
| <b>NORMALIZATION</b>        | Semantic Normal Form is assigned by the Normal Form Generator        | semantic_normal_form populated; confidence_score set                       |
| <b>EQUIVALENCE_RESOLVED</b> | Token is placed in an equivalence class by the Equivalence Evaluator | equivalence_class_id populated; equivalence type recorded                  |
| <b>LIFECYCLE_TRANSITION</b> | Token lifecycle state changes from one state to another              | lifecycle_state updated; prior state preserved in input_state snapshot     |
| <b>COMPLIANCE_CHECK</b>     | Token evaluated against a compliance framework                       | compliance_tags updated; evaluation result recorded                        |
| <b>INTEROP_MAPPING</b>      | Token mapped to or from an external system token                     | interoperability_mappings updated; target system and mapping type recorded |
| <b>HUMAN_REVIEW</b>         | Manual intervention by a semantic analyst or governance role         | Variable; depends on the nature of the review action                       |
| <b>TRIGGER FIRED</b>        | A semantic trigger evaluated as true and dispatched an action        | trigger_set updated; action dispatched; action outcome recorded            |
| <b>TRUST_SCORE_UPDATE</b>   | Token trust score is recalculated based on provenance chain changes  | trust_score updated; calculation basis recorded                            |
| <b>CONFLICT_DETECTED</b>    | Conflicting SNFs detected for the same canonical token across fields | lifecycle_state set to CONTESTED; conflict details recorded                |



| Event Type                  | Trigger Condition                                          | Key State Change                                                     |
|-----------------------------|------------------------------------------------------------|----------------------------------------------------------------------|
| <b>FIELD_VERSION_UPDATE</b> | The governing field version changes; token binding updated | semantic_field_id version component updated; prior binding preserved |

## 6.4 Provenance Authority

Not all actors may record provenance events with equal standing. The framework distinguishes between *trusted authorities* — principals whose provenance assertions are accepted by the SRE without further validation — and *untrusted actors* — principals whose assertions require countersignature by a trusted authority before they are recorded. The **Authority Registry** is a centralized catalog of all principals recognized as trusted authorities, together with the scope of their authority (i.e., which event types and which semantic fields they are authorized to assert).

Trusted authorities are classified into three tiers. *System Principals* are the automated components of the SRE itself (the Field Classifier, Normal Form Generator, Equivalence Evaluator, etc.); they are trusted for events within their functional scope. *Governance Roles* are human actors (Field Stewards, Semantic Governance Board members, designated Semantic Analysts) who have been granted authority over one or more fields and event types. *External Certification Bodies* are external organizations (regulatory bodies, standards organizations, accredited auditors) whose attestations are recognized as authoritative for specific compliance frameworks.

Every provenance event must carry an `attestation_signature`: a cryptographic signature produced by the acting authority's private key over the serialized event record (excluding the signature field itself). The signature is verified by the SRE before the event is appended to the provenance chain; events whose signatures are invalid or whose signing key is not registered in the Authority Registry for the applicable event type are rejected and recorded as `INVALID_ASSERTION` events in a separate audit log. This cryptographic attestation mechanism ensures that the provenance chain is tamper-evident: any modification to a recorded event will invalidate its signature, making the tampering immediately detectable.

## 6.5 Provenance Querying and Audit

The provenance chain is designed to support efficient querying for a broad range of audit and diagnostic use cases. The **Provenance Query Interface (PQI)** is a defined API that exposes the following query capabilities: point-in-time reconstruction, full chain retrieval, event type filtering, actor filtering, time range filtering, and provenance diffing. All queries are read-only; the PQI provides no write operations. Write access to the provenance chain is exclusively through the SRE's event recording mechanism, subject to the authority and attestation constraints described above.

Point-in-time reconstruction is the capability to answer the question: "What was the semantic state of token X at time T?" This is accomplished by replaying the provenance chain up to the event immediately preceding time T and deriving the token state at that point. Point-in-time reconstruction is essential for regulatory audit scenarios in which the regulator requires evidence of how a token was interpreted at the time a report was submitted, rather than how it is interpreted today.

Provenance diffing is the capability to compare two snapshots of the same token's provenance chain — for example, the state at report submission time versus the current state — and produce a structured description of all changes that occurred between those two points. Provenance diffing is useful for diagnosing semantic drift: the phenomenon by which a token's meaning gradually changes over time in ways that may not be immediately obvious but that can have material downstream effects.

## 6.6 Provenance and Trust Scoring

Each semantic token carries a `trust_score` — a decimal value between 0.0 and 1.0 — that reflects the overall trustworthiness of the token's semantic state based on the quality and depth of its provenance chain. The trust score is not a static value; it is recomputed whenever the provenance chain is updated and is recorded as a `TRUST_SCORE_UPDATE` event.

T

he trust score computation is a weighted function of three factors: (a) *Authority Quality*, which reflects the tier of the authorities that signed the provenance events (Tier 3 External Certification Bodies contribute the highest weight, Tier 1 System Principals contribute the lowest); (b) *Provenance Depth*, which reflects the number and completeness of provenance events relative to the expected provenance chain for a token of this type and lifecycle state (a token that has been through all required steps — creation, field binding, normalization, equivalence resolution, compliance check — has a higher depth score than one that has only been created); and (c) *Normalization Confidence*, which is the confidence score assigned by the Normal Form Generator at normalization time. Tokens with trust scores below a configurable minimum threshold (default: 0.70) are flagged with a LOW\_TRUST\_SCORE warning tag and may not be used in high-assurance downstream processes.

## 7. Semantic Lifecycle

---

### 7.1 Overview of the Semantic Lifecycle Model

Every semantic token is subject to a defined lifecycle: a finite set of operational states through which the token may transition over its existence, governed by a set of transition rules, guard conditions, and policy constraints. The lifecycle model is not an external label applied to tokens by convention; it is a first-class attribute of every semantic token, encoded in the `lifecycle_state` field and enforced by the SRE. No consuming system may rely on a token without first inspecting its lifecycle state; tokens in non-ACTIVE states carry explicit restrictions on their permissible uses.

The lifecycle model serves several essential governance purposes. First, it provides a disciplined mechanism for controlling the operational availability of tokens: only tokens in the ACTIVE state are fully available for all downstream uses; tokens in other states are subject to defined restrictions. Second, it provides a mechanism for managing the obsolescence of tokens: as meanings evolve, tokens are transitioned through DEPRECATED and RETIRED states in a controlled manner rather than abruptly removed, giving downstream systems time to adapt. Third, it provides a mechanism for managing quality and compliance exceptions: tokens that fail normalization, encounter equivalence conflicts, or violate compliance requirements are moved to QUARANTINED or CONTESTED states, suspending their downstream use until the issue is resolved.

## 7.2 Lifecycle States

| State                 | Description                                                                                                                               | Downstream Use Permitted                                                |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| <b>DRAFT</b>          | Token created; awaiting field binding and normalization. Token is not yet semantically resolved; its meaning is undefined.                | No downstream use permitted                                             |
| <b>PENDING_REVIEW</b> | Token normalized (possibly with low confidence); awaiting governance approval. Token has a candidate SNF but it has not been approved.    | Read-only; analytics only; not for compliance-sensitive use             |
| <b>ACTIVE</b>         | Token fully resolved, approved, and in operational use. All fields fully populated; compliance tags current; trust score above threshold. | Full downstream use permitted                                           |
| <b>DEPRECATED</b>     | Token superseded by a newer version or SNF; remains resolvable for legacy compatibility. New tokens should not be assigned to this SNF.   | Read-only; legacy resolution only; must not be used in new integrations |
| <b>RETIRED</b>        | Token permanently decommissioned; no longer resolves for downstream use. Provenance chain preserved indefinitely for audit.               | No downstream use permitted; provenance accessible for audit            |
| <b>QUARANTINED</b>    | Token flagged for compliance or quality issues; all downstream use suspended pending review and remediation.                              | No downstream use permitted; visible only to governance actors          |
| <b>CONTESTED</b>      | Token subject to active equivalence or meaning dispute; use suspended pending dispute resolution by the Semantic Governance Board.        | No downstream use permitted; visible only to governance actors          |

### 7.3 Lifecycle Transitions

Not all state transitions are valid. The lifecycle model defines a directed state graph in which only specific transitions are permitted. Attempting an invalid transition — for example, transitioning directly from DRAFT to ACTIVE without passing through PENDING\_REVIEW — results in a transition rejection event. The valid transitions are:

- **DRAFT → PENDING\_REVIEW:** Triggered automatically by the SRE upon successful normalization (even if with low confidence). Guard: normalization pipeline has completed without fatal error.
- **PENDING\_REVIEW → ACTIVE:** Triggered by governance approval (Field Steward or delegate). Guard: trust score  $\geq$  minimum threshold; all required compliance tags present; no open equivalence disputes.
- **PENDING\_REVIEW → QUARANTINED:** Triggered by normalization confidence below quarantine threshold or compliance violation detected. Guard: none; quarantine takes immediate effect.
- **ACTIVE → DEPRECATED:** Triggered by field version increment, superseding token creation, or governance decision. Guard: successor token or field version is ACTIVE.
- **ACTIVE → QUARANTINED:** Triggered by trigger dispatch, compliance violation, or governance override. Guard: none; quarantine takes immediate effect.
- **ACTIVE → CONTESTED:** Triggered by detection of irreconcilable SNF conflict. Guard: conflict must be formally logged in the conflict registry.
- **DEPRECATED → RETIRED:** Triggered by retention policy expiry or explicit governance action. Guard: no active downstream dependencies registered against this token.
- **QUARANTINED → PENDING\_REVIEW:** Triggered by remediation completion and governance clearance. Guard: remediation record in provenance chain; clearance by authorized governance role.
- **QUARANTINED → RETIRED:** Triggered by determination that the token is irreparably invalid. Guard: governance approval by Semantic Governance Board.
- **CONTESTED → ACTIVE:** Triggered by successful dispute resolution. Guard: resolution record in provenance chain; approval by Semantic Governance Board.

- **CONTESTED → RETIRED:** Triggered by determination that the token cannot be resolved. Guard: governance approval by Semantic Governance Board.

## 7.4 Lifecycle Policies

The lifecycle model is governed by a set of formal policies that define mandatory behaviors at each state and during each transition. Lifecycle policies are defined per semantic field by the field steward and approved by the Semantic Governance Board. They are versioned and subject to the same governance discipline as field definitions. The following policy categories are mandatory for every registered semantic field:

*Retention Policies* define how long a token may remain in each state before a forced transition is initiated. For example, a retention policy might specify that tokens may not remain in PENDING\_REVIEW for more than 30 calendar days without triggering an escalation; or that tokens in DEPRECATED state must be transitioned to RETIRED within 18 months of the deprecation date.

*Archival Policies* define what provenance data is preserved after a token reaches the RETIRED state. The minimum archival requirement is that the complete provenance chain is preserved in read-only archival storage for a period of not less than seven years (or the applicable regulatory retention period if longer). SNF and compliance tag data must also be preserved. The archived record must remain queryable through the PQI for the full archival period.

*Review Policies* define mandatory periodic review intervals for ACTIVE tokens in sensitive semantic fields. For example, tokens in the "Regulatory Reporting" field may be required to undergo compliance re-evaluation every 90 days; tokens in the "Customer Identity" field may require privacy compliance re-evaluation annually or upon any change to the applicable privacy regulation. Review events are scheduled automatically by the lifecycle management subsystem and are recorded as COMPLIANCE\_CHECK events in the provenance chain.

## 7.5 Lifecycle and Downstream Systems

The lifecycle state of a semantic token is not merely an internal framework concern; it is a signal that must be propagated to and respected by all consuming downstream systems. The SRE propagates lifecycle state changes to all registered downstream consumers via the **Semantic Event Bus**, a publish-subscribe messaging infrastructure that delivers structured lifecycle events in near-real time. Downstream systems are required to register their token dependencies at integration time; upon registration, they receive lifecycle events for all tokens they depend on.

Consuming systems have defined obligations based on the lifecycle state of tokens they hold. A consuming system that holds a reference to a token that has transitioned to DEPRECATED must, within the defined migration window, either migrate to the successor token or record a formal waiver approved by the field steward. A consuming system that holds a reference to a token that has transitioned to QUARANTINED must immediately suspend all operations that depend on that token's semantic interpretation. A consuming system that holds a reference to a RETIRED token must remove that reference entirely and replace it with a reference to an ACTIVE successor, or record the absence of a successor and escalate to the Semantic Governance Board.

Graceful degradation is supported for consuming systems that receive a token in DEPRECATED state. Such tokens remain resolvable; the SRE returns the DEPRECATED token's SNF alongside a pointer to the successor token and a migration advisory. Consuming systems may continue to use the DEPRECATED token for read purposes during the migration window, but may not use it to generate new outbound identifiers or regulatory reports. This grace period design balances the practical reality of migration timelines against the governance imperative of eventual consistency.

## 8. Semantic Equivalence

---

### 8.1 Definition of Semantic Equivalence

#### Formal Definition — Semantic Equivalence

Two semantic tokens  $ST_A$  and  $ST_B$  are **semantically equivalent** if and only if: (1) they are bound to the same semantic field and field version, (2) they resolve to the same underlying concept as determined by the governing ontology of that field, and (3) the equivalence is asserted with a confidence level at or above the applicable threshold by an authority recognized in the Authority Registry.

**Key constraint:** Semantic equivalence is field-scoped, version-scoped, and confidence-bounded. It is not a universal, context-independent property.

The field-scoping and version-scoping of semantic equivalence are not mere formalities; they reflect a fundamental property of meaning itself. The same pair of tokens may be equivalent in one semantic field and non-equivalent in another. "MI" and "myocardial infarction" are equivalent in the healthcare field but carry entirely different meanings in the intelligence field (where "MI" may refer to "military intelligence"). Similarly, two tokens that were equivalent under field version 2.1 may no longer be equivalent under field version 2.2 if the ontology was revised to distinguish between concepts that were previously merged. Version-scoped equivalence ensures that historical equivalence relationships are preserved intact even as the current-version relationships evolve.



## 8.2 Equivalence Classes

Semantic equivalence is managed through the **Equivalence Class Registry**, a centralized catalog of equivalence classes — sets of semantic tokens that are interchangeable within a defined semantic field and context. Equivalence classes are governed artifacts: they are versioned, owned by a designated authority, and subject to the same lifecycle model as tokens.

| Field                 | Type                | Description                                                                                                               |
|-----------------------|---------------------|---------------------------------------------------------------------------------------------------------------------------|
| class_id              | UUID                | Globally unique identifier for this equivalence class                                                                     |
| semantic_field_id     | URI + Version       | The semantic field and version within which this equivalence class is defined                                             |
| canonical_concept_uri | URI                 | The concept URI in the governing ontology that all members of this class represent; the "ground truth" of the equivalence |
| member_tokens         | Array<token_id>     | Set of semantic token IDs that are members of this equivalence class                                                      |
| equivalence_type      | Enum                | Type of equivalence: STRICT   FUNCTIONAL   APPROXIMATE   HISTORICAL                                                       |
| confidence_threshold  | Decimal (0.0–1.0)   | Minimum confidence score required for a token to assert membership in this class                                          |
| governing_authority   | Principal reference | The authority responsible for managing this equivalence class                                                             |
| class_version         | String (SemVer)     | Version of this equivalence class definition; incremented when membership or concept URI changes                          |
| uncertainty_bounds    | Object (optional)   | For APPROXIMATE equivalence classes: quantified uncertainty bounds and applicable use case scope                          |

### 8.3 Types of Semantic Equivalence

The framework recognizes four distinct types of semantic equivalence, each appropriate for a different context and each carrying different implications for downstream use:

**Strict Equivalence** is the strongest form: two tokens are strictly equivalent if they resolve to identical SNFs — the same concept URI, the same preferred label, and the same field-specific attributes. Strict equivalence is bidirectional and transitive: if A is strictly equivalent to B, and B is strictly equivalent to C, then A is strictly equivalent to C. Strict equivalence supports lossless substitution; a consuming system may replace any token in a strict equivalence class with any other member without any loss of semantic fidelity.

**Functional Equivalence** is a weaker form: two tokens resolve to different SNFs but are operationally interchangeable in a specific pipeline context. For example, in a product fulfillment context, "Widget Model A" and "Widget Model A (Revised)" may be functionally equivalent — the fulfillment system treats them identically — even though their governing ontology distinguishes between them. Functional equivalence is context-scoped: it is defined relative to a specific processing context and does not imply equivalence in other contexts. It is recorded in the Equivalence Class Registry with a context qualifier that specifies the scope of functional interchangeability.

**Approximate Equivalence** is the weakest form that still supports meaningful comparison: two tokens are approximately equivalent if they are sufficiently similar for a defined use case, with explicitly quantified uncertainty bounds. Approximate equivalence is appropriate for probabilistic matching scenarios — entity resolution, deduplication, fuzzy classification — where the governing ontology does not provide a definitive equivalence determination. Approximate equivalence records include a confidence score and an uncertainty quantification that consuming systems must incorporate into their downstream processing.

**Historical Equivalence** is a temporal form: two tokens that were once strictly or functionally equivalent but have since diverged as the governing ontology or field definition evolved. Historical equivalence is essential for provenance and audit: when a regulator asks whether two tokens referred to the same concept at the time of a historical submission, historical equivalence records provide the answer. Historical equivalence is always time-bounded, with a `valid_from` and `valid_until` timestamp pair.

## 8.4 Equivalence Resolution Algorithm

The Equivalence Evaluator component of the SRE applies the following five-step algorithm to determine the equivalence relationship between two or more canonical tokens in a specified semantic field context:

6. **SNF Resolution:** Each input canonical token is resolved to its SNF within the specified semantic field and field version, using the normalization pipeline described in Section 5.3.
7. **Direct Class Lookup:** The resolved SNFs are compared against the Equivalence Class Registry. If all input SNFs appear as members of the same equivalence class of type STRICT or FUNCTIONAL (and the applicable context matches), the algorithm returns EQUIVALENT with the class ID and confidence score as evidence.
8. **Ontology Traversal:** If no direct class membership is found, the Equivalence Evaluator performs ontology traversal — following `skos:exactMatch`, `owl:sameAs`, `skos:broadMatch`, and `skos:narrowMatch` relationships in the governing ontology — to determine whether an indirect equivalence path exists between the candidate concept URIs.
9. **Approximate Matching:** If ontology traversal does not yield a definitive result, the Equivalence Evaluator applies approximate matching — using configurable similarity metrics (e.g., label similarity, attribute overlap, graph distance) — to compute an approximate equivalence score.
10. **Result Emission:** The algorithm returns a structured equivalence result containing: the equivalence type (EQUIVALENT / APPROXIMATE / NOT\_EQUIVALENT), the confidence score, the resolution path (direct class, ontology traversal, or approximate match), and any applicable class membership evidence. The result is recorded in each participating token's provenance chain as an EQUIVALENCE\_RESOLVED event.

## 8.5 Equivalence Disputes

Semantic equivalence disputes arise when two systems or authorities assert contradictory equivalence relationships for the same pair of tokens. Disputes may occur due to ontology version mismatches, context misspecification, domain expert disagreement, or emerging classification standards that have not yet been fully incorporated into the governing ontology. The framework provides a formal dispute management mechanism to ensure that disputes are logged, escalated, and resolved in a governed manner rather than silently resolved by whichever system's assertion was received last.

When an equivalence dispute is detected — identified by the Equivalence Evaluator when it receives two conflicting equivalence assertions for the same token pair and field — both disputing tokens are transitioned to the `CONTESTED` lifecycle state, a `CONFLICT_DETECTED` event is recorded in each token's provenance chain, and a dispute record is created in the Equivalence Dispute Log. The dispute record includes the identities of both asserting authorities, the conflicting assertions, the timestamp of each assertion, and the field and field version context. The Semantic Governance Board is notified and must resolve the dispute within the timeframe specified in the relevant lifecycle policy. The resolution — which takes the form of a governance-approved equivalence determination — is recorded in the Equivalence Class Registry and in each token's provenance chain as a `DISPUTE_RESOLVED` event.

## 8.6 Equivalence Maintenance

Equivalence classes are not static; they must be actively maintained as semantic fields evolve. When a field version is incremented, the Equivalence Evaluator revalidates all equivalence class memberships under the new field version. Members whose SNFs have changed as a result of the field version update are removed from their prior-version equivalence classes and re-evaluated for membership in the new-version equivalence classes. Prior-version equivalence class records are preserved in `RETIRED` state for historical audit purposes.

The impact of SNF changes on existing equivalence class memberships is categorized as one of three levels. *Non-impacting changes* (e.g., preferred label changes that do not alter the concept URI) preserve all existing class memberships unchanged. *Partially impacting changes* (e.g., the addition of new disambiguation qualifiers that narrow a previously broad concept) trigger selective removal of affected members and re-evaluation. *Fully impacting changes* (e.g., concept URI changes resulting from ontology restructuring) invalidate all memberships for the affected tokens and require full re-evaluation; affected tokens are transitioned to `PENDING_REVIEW` during the re-evaluation period.

## 9. Semantic Triggers

---

### 9.1 Definition and Purpose

#### Formal Definition — Semantic Trigger

A **Semantic Trigger** is a declarative, governed rule that specifies a condition over token semantic state and context, and an action to be executed when that condition is evaluated as true. Triggers are the mechanism by which the semantic tokenization framework achieves event-driven, automated semantic governance without embedding semantic logic in consuming applications.

Semantic triggers decouple semantic state management from consuming application logic — a critical architectural property in large-scale enterprise deployments where hundreds of applications may consume semantic tokens from a shared registry. Without triggers, each consuming application would need to implement its own logic for detecting and responding to semantic state changes; in practice, this leads to inconsistent responses, missed events, and duplicated governance logic scattered across the application landscape. With triggers, semantic state management logic is centralized, governed, and auditable: it resides in the Trigger Registry, is managed by designated authorities, and produces provenance events for every firing.

Triggers are governed artifacts with their own lifecycle: they are created in DRAFT state, undergo governance review, are approved into ACTIVE state, and may be DEPRECATED or RETIRED as the semantic governance requirements they serve evolve. A trigger that is in DEPRECATED or RETIRED state will not be evaluated by the Trigger Dispatcher, even if its condition would otherwise evaluate as true. This lifecycle discipline ensures that obsolete trigger logic does not produce spurious actions in the system.

## 9.2 Trigger Architecture

Every trigger in the Trigger Registry is defined by four components:

- **Condition:** A predicate expression written in the Trigger Condition Language (Section 9.3) that is evaluated against the current token state and resolution context. A trigger fires if and only if its condition evaluates to *true*.
- **Action:** The system action to be executed when the trigger fires, specified as one of the defined Action Types (Section 9.4) with applicable parameters.
- **Authority:** The governance role or system principal authorized to define and activate this trigger. Only authorities with field-scoped trigger definition rights may create triggers for tokens in that field.
- **Scope:** The set of semantic fields, token types, and lifecycle states to which this trigger applies. The Trigger Dispatcher evaluates a trigger against an incoming token state change event only if the token falls within the trigger's scope.

The **Trigger Evaluation Engine (TEE)** is an event-driven subsystem of the SRE that receives token state change events from the SRE's internal event bus and evaluates all in-scope, ACTIVE triggers against each event. The TEE is designed for low-latency evaluation; trigger condition evaluation must complete within a configurable timeout (default: 500 milliseconds). Triggers that time out during condition evaluation are logged as TRIGGER\_TIMEOUT events and re-evaluated at the next scheduled evaluation cycle. The TEE supports both synchronous trigger evaluation (for triggers that must complete before the SRE emits the resolved token) and asynchronous trigger evaluation (for triggers whose actions are independent of the token resolution result).

### 9.3 Trigger Condition Language

The Trigger Condition Language (TCL) is a domain-specific declarative expression language designed for expressing predicates over semantic token state. TCL expressions are evaluated by the Trigger Evaluation Engine at runtime. The language is intentionally constrained — it is not Turing-complete — to ensure that trigger conditions can be statically analyzed, automatically tested, and efficiently evaluated without the risk of infinite loops or resource exhaustion.

TCL supports the following predicate classes:

| Predicate                                         | Description                                                                         | Example                                                  |
|---------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------|
| <code>field_is(field_id)</code>                   | True if the token is bound to the specified semantic field                          | <code>field_is("urn:field:customer-identity")</code>     |
| <code>lifecycle_state_is(state)</code>            | True if the token's current lifecycle state matches the specified value             | <code>lifecycle_state_is(DEPRECATED)</code>              |
| <code>equivalence_class_contains(class_id)</code> | True if the token is a member of the specified equivalence class                    | <code>equivalence_class_contains("class:cust-01")</code> |
| <code>provenance_actor_is(actor_id)</code>        | True if the most recent provenance event was asserted by the specified actor        | <code>provenance_actor_is("sys:norm-engine")</code>      |
| <code>confidence_below(threshold)</code>          | True if the token's normalization confidence score is below the specified threshold | <code>confidence_below(0.85)</code>                      |
| <code>snf_matches(concept_uri)</code>             | True if the token's SNF concept URI                                                 | <code>snf_matches("snomed:22298006")</code>              |

| Predicate                                         | Description                                                                             | Example                                          |
|---------------------------------------------------|-----------------------------------------------------------------------------------------|--------------------------------------------------|
|                                                   | matches the specified URI                                                               |                                                  |
| <code>compliance_tag_missing(framework_id)</code> | True if the token has no compliance tag for the specified framework                     | <code>compliance_tag_missing("GDPR-Art5")</code> |
| <code>trust_score_below(threshold)</code>         | True if the token's trust score is below the specified threshold                        | <code>trust_score_below(0.70)</code>             |
| <code>days_in_state(n)</code>                     | True if the token has been in its current lifecycle state for more than n calendar days | <code>days_in_state(30)</code>                   |

TCL predicates may be combined using Boolean operators (AND, OR, NOT) and grouped using parentheses. Temporal operators (WITHIN, SINCE, BEFORE) allow conditions to be expressed relative to specific events in the provenance chain. Universal and existential quantifiers (ALL, ANY) allow conditions to range over sets of related tokens (e.g., all tokens in the same equivalence class).



## 9.4 Trigger Action Types

When a trigger fires, it dispatches one or more actions of defined types. Each action type has a defined execution contract: the SRE guarantees that NOTIFY, QUARANTINE, and LIFECYCLE\_TRANSITION actions are executed synchronously (before the token state update is committed); ROUTE, ENRICH, REPLICATE, and ESCALATE actions are executed asynchronously (after the token state update is committed, with at-least-once delivery guarantees).

| Action Type       | Description                                                                                                                                                                                                                 | Execution Mode |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| <b>NOTIFY</b>     | Send a structured alert to a designated role, team, or system endpoint. Alert includes token ID, trigger ID, condition that fired, and current token state.                                                                 | Asynchronous   |
| <b>ROUTE</b>      | Forward the token (or a token update event) to a specified processing pipeline, queue, or service endpoint. Used to direct tokens to specialist processing based on semantic state.                                         | Asynchronous   |
| <b>ESCALATE</b>   | Initiate a formal governance review workflow, creating a work item in the Semantic Governance Board's workflow system with the trigger context and token state as input.                                                    | Asynchronous   |
| <b>QUARANTINE</b> | Transition the token to QUARANTINED lifecycle state, recording a LIFECYCLE_TRANSITION provenance event with the trigger ID as the authority reference.                                                                      | Synchronous    |
| <b>ENRICH</b>     | Invoke a designated enrichment service to augment the token's metadata (e.g., add domain-specific attributes, update field-specific annotations). Enrichment results are appended to the token schema and provenance chain. | Asynchronous   |
| <b>REPLICATE</b>  | Push a token update notification and updated token metadata to all registered downstream semantic stores and consuming system subscriptions.                                                                                | Asynchronous   |
| <b>EXPIRE</b>     | Initiate a lifecycle transition to DEPRECATED or RETIRED, subject to the transition guard conditions defined in Section 7.3.                                                                                                | Synchronous    |

## 9.5 Trigger Governance

Triggers are subject to the same lifecycle governance model as tokens. A newly created trigger is in DRAFT state and is evaluated by the Semantic Governance Board before it is approved into ACTIVE state. The approval process includes: syntactic validation of the TCL condition expression; semantic review of the condition to confirm it correctly captures the intended governance intent; conflict analysis against all existing ACTIVE triggers to identify potential conflicts; and authority verification to confirm that the proposing principal has field-scoped trigger definition rights.

Trigger conflicts arise when two or more triggers fire on the same token state change event and their actions are contradictory or mutually exclusive. For example, a trigger that fires QUARANTINE and a trigger that fires EXPIRE on the same event would conflict because QUARANTINE and RETIRE are valid but distinct lifecycle states with different downstream implications. Trigger conflicts are resolved through a defined priority order: QUARANTINE actions take precedence over all other actions; ESCALATE actions take precedence over NOTIFY and ROUTE actions; higher-tier authority triggers take precedence over lower-tier authority triggers. When a conflict is resolved by priority, the losing trigger's action is suppressed, and a TRIGGER\_CONFLICT\_RESOLVED event is recorded in the token's provenance chain.

## 9.6 Example Trigger Scenarios

### Example Trigger 1 — Deprecation Migration Notification

**Condition:** `field_is("urn:field:customer-identity") AND lifecycle_state_is(DEPRECATED)`

**Action:** `NOTIFY(target="crm-integration-team@org.internal", message="Customer Identity token deprecated; 30-day migration window active") AND ROUTE(target="queue:migration-workflow", payload=token_metadata)`

**Purpose:** Ensures that when any Customer Identity field token is deprecated, the CRM integration team is notified immediately and a migration workflow is initiated automatically, with a 30-day window before the token is retired.

### Example Trigger 2 — Low-Confidence Quarantine

**Condition:** confidence\_below(0.85) AND NOT lifecycle\_state\_is(QUARANTINED)

**Action:** QUARANTINE AND ESCALATE(target="semantic-governance-board", priority=HIGH)

**Purpose:** Automatically quarantines any token whose normalization confidence falls below 0.85 (the configured threshold) and escalates it to the Semantic Governance Board for human review, preventing low-confidence tokens from entering downstream processes undetected.

### Example Trigger 3 — Functional Equivalence Replication

**Condition:** equivalence\_class\_contains(ANY) AND lifecycle\_state\_is(ACTIVE)

**Action:** REPLICATE(target="all-registered-consumers", include\_equivalence\_class\_data=true)

**Purpose:** When two tokens in different fields are asserted as functionally equivalent and both are ACTIVE, the equivalence mapping is replicated to all registered downstream consumers so that they may immediately update their local token resolution caches without waiting for a scheduled sync cycle.

## 10. Semantic Compliance

---

### 10.1 Compliance as a Semantic Concern

Regulatory and policy frameworks impose requirements on data that are fundamentally semantic in nature: they mandate not merely that data is stored in a particular format or transmitted using a particular protocol, but that data means what it is represented as meaning, that its lineage can be traced, and that its use is consistent with its stated purpose. The GDPR's accuracy principle requires that personal data be "accurate and, where necessary, kept up to date" — this is a semantic requirement, not a structural one. HIPAA's de-identification standards specify that certain concepts must be transformed to prevent re-identification — again, a requirement that operates at the level of meaning rather than form. PCI DSS's tokenization requirements mandate that cardholder data be replaced by tokens that have "no exploitable value" — a semantic property of the token that cannot be satisfied by canonical tokenization alone.

**Semantic Compliance**, as defined in this framework, is the property of a semantic token whereby its meaning, field binding, provenance chain, and lifecycle state jointly satisfy all applicable governance policies and regulatory obligations. Semantic compliance is not a binary attribute; it is a structured evaluation result that maps each applicable compliance framework to a satisfaction status, supported by evidence and recorded in a tamper-evident manner. The compliance tag mechanism described in this section provides the data structure for representing semantic compliance, and the compliance evaluation workflow describes the process by which it is assessed.

## 10.2 Compliance Tag Schema

Each semantic token carries a set of **Compliance Tags** — one per applicable compliance framework — that record the results of compliance evaluation. Compliance tags are structured objects with the following schema:

| Field               | Type                           | Description                                                                                                          |
|---------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------|
| framework_id        | String (URI)                   | Unique identifier of the compliance framework (e.g., "urn:compliance:gdpr-art5", "urn:compliance:hipaa-safe-harbor") |
| framework_version   | String (SemVer)                | Version of the compliance framework definition against which this evaluation was performed                           |
| requirement_id      | String                         | Specific requirement within the framework that this tag addresses (e.g., "Art5.1(d) — Accuracy")                     |
| status              | Enum                           | Evaluation result: SATISFIED   PENDING   VIOLATED   EXEMPT                                                           |
| assessed_at         | ISO 8601 Timestamp             | Timestamp of the most recent compliance assessment for this tag                                                      |
| assessed_by         | Principal reference            | The system principal, governance role, or external body that performed the assessment                                |
| evidence_reference  | URI (optional)                 | Reference to the provenance event, rule, or external document that constitutes evidence for the compliance status    |
| next_review_due     | ISO 8601 Date (optional)       | Date by which this compliance evaluation must be refreshed, per the applicable review policy                         |
| exemption_authority | Principal reference (optional) | For EXEMPT status: the authority that granted the exemption and the basis for the exemption                          |

Compliance tags are provenance-linked: every change to a compliance tag — whether a status update, an assessment refresh, or the addition of a new tag — is recorded as a COMPLIANCE\_CHECK event in the token's provenance chain, referencing the updated tag's content and the identity of the assessing authority. This linkage ensures that the compliance history of every token is fully traceable and that no compliance status change occurs without a corresponding audit record.

### 10.3 Compliance Frameworks Supported

The framework provides a structured compliance framework registry that defines, for each supported compliance framework, the set of requirements that are evaluated against semantic tokens, the evaluation logic for each requirement, and the mapping between token schema fields and framework requirement criteria. The registry is extensible: organizations may define and register custom compliance frameworks to express internal governance policies in the same structured format as external regulatory frameworks.

| Framework                              | Domain                           | Key Requirements Addressed                                                                                                       |
|----------------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| GDPR Article 5                         | Data Privacy (EU)                | Data accuracy (Art. 5.1(d)); purpose limitation (Art. 5.1(b)); storage limitation (Art. 5.1(e)); data minimization (Art. 5.1(c)) |
| CCPA (California Consumer Privacy Act) | Data Privacy (US)                | Purpose limitation; right to deletion; data accuracy for consumer-facing data                                                    |
| NIST SP 800-63 (Rev. 4)                | Identity Governance (US Federal) | Identity assurance levels; proofing evidence requirements; authenticator binding                                                 |
| HIPAA Safe Harbor                      | Healthcare Privacy (US)          | De-identification of 18 specified identifier categories; prohibition on re-identification                                        |
| PCI DSS v4.0                           | Payment Card Industry            | Cardholder data tokenization requirements; token irreversibility; secure token vault architecture                                |
| SOX Section 302/404                    | Financial Reporting (US)         | Data integrity for financial reporting tokens; audit trail completeness; management attestation support                          |
| Custom Internal Frameworks             | Organization-specific            | Any governance policy expressed in the compliance framework schema format                                                        |

## 10.4 Semantic Compliance Evaluation

Compliance evaluation is a multi-step process triggered by defined events in the token lifecycle. The evaluation is not a single one-time assessment; it is a continuous, policy-driven activity that ensures tokens remain compliant throughout their operational lifetime as regulations evolve, token states change, and new compliance frameworks are adopted.

Compliance evaluation is triggered by the following events: (a) token creation — all applicable frameworks are evaluated immediately upon the token reaching `PENDING_REVIEW` state; (b) lifecycle transition — every transition of the token's lifecycle state triggers re-evaluation of all applicable frameworks, since different states have different compliance implications; (c) scheduled review — as specified in the lifecycle policy's review intervals, compliance is re-evaluated on a calendar schedule; and (d) on-demand evaluation — any governance actor with appropriate authority may initiate an immediate compliance evaluation on any token.

The evaluation steps are as follows: First, the Compliance Checker identifies all applicable compliance frameworks by intersecting the token's semantic field bindings and token type with the framework registry's applicability rules. Second, for each applicable framework and each requirement within that framework, the Compliance Checker evaluates the requirement's predicate against the token's current state — its SNF, provenance chain, lifecycle state, trust score, and existing compliance tags. Third, the Compliance Checker updates the relevant compliance tag with the evaluation result, the assessing authority's identity, and the timestamp. Fourth, a compliance report artifact is generated summarizing the full evaluation results. Fifth, a `COMPLIANCE_CHECK` provenance event is appended to the token's provenance chain, referencing the compliance report artifact.

## 10.5 Compliance Violations and Remediation

When a `VIOLATED` status is set on any compliance tag, the semantic trigger system is notified and evaluates all triggers with conditions that include compliance violation predicates. In the default configuration, a compliance violation triggers automatic `QUARANTINE` of the affected token, a `NOTIFY` action to the designated compliance officer, and an `ESCALATE` action to the Semantic Governance Board. The token remains in `QUARANTINED` state until a remediation action is completed and a governance actor with appropriate authority clears the quarantine.

Remediation actions available for compliance violations include: *re-normalization* — re-running the normalization pipeline with updated rules that satisfy the violated requirement; *provenance augmentation* — adding provenance events that provide the missing evidence required by the framework (e.g., recording the authority attestation that was absent at the time of original assessment); *lifecycle transition* — transitioning the token to a state in which the violated requirement no longer applies (e.g., transitioning a token with a GDPR storage limitation violation to RETIRED if the token has exceeded its permitted retention period); and *human review* — engaging a human governance actor to assess the violation and determine the appropriate remediation path.

Compliance violations that cannot be remediated within the timeframe specified in the applicable lifecycle policy are escalated to the Semantic Governance Board with the full provenance chain, compliance tag history, and remediation attempt record as supporting documentation. The Governance Board may authorize an extended remediation window, approve an exemption with documented justification, or mandate token retirement as the resolution.

## **10.6 Compliance Reporting**

The framework provides three categories of compliance reporting artifacts, each designed for a specific audience and use case. All compliance reporting artifacts are tamper-evident: they are cryptographically signed by the Semantic Resolution Engine at generation time, ensuring that the reported compliance state cannot be altered after the fact without detection.

*Point-in-Time Compliance Snapshots* are exportable reports that capture the compliance posture of all tokens in a specified semantic field at a given point in time. They answer the question: "As of date T, what was the compliance status of every token in field F?" These reports are the primary artifact used to respond to regulatory examinations and to demonstrate compliance at the time of a historical filing or submission. They are generated from the provenance chain using the point-in-time reconstruction capability of the Provenance Query Interface.

*Compliance Trend Reports* are time-series reports that track the aggregate compliance posture of a semantic field over a specified time range. They show the proportion of tokens in each compliance status (SATISFIED, PENDING, VIOLATED, EXEMPT) at each point in time, enabling governance actors to identify trends — improving compliance posture, deteriorating compliance posture, or the impact of specific remediation initiatives — and to report on progress to executive stakeholders.

*Audit-Ready Artifacts* are structured packages suitable for submission to regulatory bodies or external auditors. They include: the point-in-time compliance snapshot for the relevant reporting period; the full provenance chain for each token referenced in the snapshot; the compliance framework version used for evaluation; the identity and attestation signatures of all assessing authorities; and a machine-verifiable cryptographic integrity proof over the entire package. These artifacts are designed to satisfy the evidentiary standards of major regulatory frameworks including GDPR, HIPAA, SOX, and PCI DSS.

## **11. Semantic Interoperability**

---

### **11.1 The Interoperability Problem**

The challenge of semantic interoperability is one of the most persistent and costly problems in enterprise data architecture. Different systems, developed at different times by different teams using different modeling conventions, represent the same real-world entities and concepts using different tokens, labels, URIs, and schemas. A customer who is "CUST-00124891" in the CRM system is "ACC-2019-87432" in the billing system and "EMP-SPOUSE-443" in the HR benefits system. A product that is "SKU-87-ALPHA" in the procurement system is "PROD.0087.A" in the supply chain system and "ITEM\_87A" in the e-commerce platform. The manual reconciliation of these identities across systems is expensive, error-prone, and a persistent source of data quality failures.

Canonical tokenization addresses part of this problem by normalizing the structural form of tokens within a single governance domain. But canonical normalization does not, by itself, establish meaning-preserving mappings between the token spaces of different organizations or different ontological frameworks. Even if both systems agree on the canonical form of a token, they may still disagree on what that token means within their respective domains. Semantic interoperability is the capability to exchange semantic tokens across system and organizational boundaries while preserving — not merely approximating — the meaning that the source system intended to convey.



## 11.2 Interoperability Mapping Architecture

Each semantic token carries an `interoperability_mappings` field containing zero or more mapping entries. Each entry specifies a target system, the token identifier used by that system to refer to the same concept, the type of mapping relationship, and the authority and validity period of the mapping. Mappings are bidirectional: inbound mappings specify how external tokens are translated into the local token's SNF when received from the target system; outbound mappings specify how the local SNF is translated into the target system's token representation when sent to that system.

| Field                              | Type                                | Description                                                                                                  |
|------------------------------------|-------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <code>target_system_id</code>      | String (URI)                        | Unique identifier of the external system or organization to which this mapping applies                       |
| <code>target_schema_version</code> | String (SemVer)                     | Version of the target system's token schema at the time this mapping was established                         |
| <code>target_token_id</code>       | String                              | The token identifier used by the target system to refer to the same concept                                  |
| <code>mapping_type</code>          | Enum                                | EXACT   BROADER   NARROWER   RELATED   CUSTOM                                                                |
| <code>mapping_direction</code>     | Enum                                | INBOUND   OUTBOUND   BIDIRECTIONAL                                                                           |
| <code>confidence</code>            | Decimal (0.0–1.0)                   | Confidence level of this mapping relationship                                                                |
| <code>mapping_authority</code>     | Principal reference                 | The authority that established and is responsible for this mapping                                           |
| <code>valid_from</code>            | ISO 8601 Date                       | Date from which this mapping is effective                                                                    |
| <code>valid_until</code>           | ISO 8601 Date (optional)            | Date until which this mapping is effective; null for open-ended mappings                                     |
| <code>transformation_logic</code>  | URI or inline expression (optional) | For CUSTOM and RELATED mapping types: reference to or inline expression of the transformation logic required |

### 11.3 Mapping Types

The framework defines five mapping types, borrowed and adapted from the SKOS mapping properties vocabulary. Each type carries a defined set of semantic implications for consuming systems:

- **EXACT:** A one-to-one, lossless mapping. The source and target tokens refer to the same concept with no ambiguity or scope difference. EXACT mappings support full substitution: a consuming system may use either token interchangeably. EXACT mappings correspond to `skos:exactMatch` in the SKOS vocabulary.
- **BROADER:** The internal (source) concept is more specific than the external (target) concept. The source is a subtype or specialization of the target. A consuming system that receives a BROADER-mapped token must be aware that the source concept is narrower than what it may be expecting. BROADER mappings correspond to `skos:narrowMatch` from the external system's perspective.
- **NARROWER:** The internal (source) concept is broader than the external (target) concept. The target is a subtype or specialization of the source. A consuming system that receives a NARROWER-mapped token receives more specificity than the source provided; the receiving system may benefit from the additional precision but must not assume that all tokens mapped via this relationship have the same degree of specificity. NARROWER mappings correspond to `skos:broadMatch` from the external system's perspective.
- **RELATED:** The source and target concepts are related but not directly equivalent; semantic transformation is required. RELATED mappings specify a transformation expression that the Semantic Translation Protocol applies when translating the token. Consuming systems must apply the transformation and understand that semantic loss or gain may occur. RELATED mappings correspond to `skos:relatedMatch`.
- **CUSTOM:** A bespoke mapping with documented transformation logic that does not fit any of the standard types. CUSTOM mappings require explicit governance review and approval before they may be used in production; they are recorded with a full description of the transformation logic and its known limitations.

## 11.4 Semantic Translation Protocol

The **Semantic Translation Protocol (STP)** is the defined procedure by which a semantic token is translated when it crosses a system boundary. The STP ensures that the receiving system receives not just a transformed token identifier but the full semantic context necessary to interpret the token correctly — including the source SNF, the applicable provenance chain, and the mapping metadata — while expressing the token in the vocabulary and schema expected by the receiving system.

The STP executes the following steps for each outbound token crossing:

11. **Source Token Validation:** Verify that the source semantic token is in ACTIVE lifecycle state.  
Tokens in any other state may not be transmitted via the STP unless the target system has explicitly declared a requirement for legacy token access.
12. **Mapping Lookup:** Look up the applicable outbound mapping for the target system in the token's `interoperability_mappings` field. If no mapping exists, the STP checks the Interoperability Mapping Registry for a registered mapping that has not yet been applied to this token, and applies it. If no mapping can be found, the STP returns a `MAPPING_NOT_FOUND` error to the requesting system.
13. **Transformation Application:** If the mapping type is RELATED or CUSTOM, apply the specified transformation logic to the source SNF to produce the target token representation. For EXACT, BROADER, and NARROWER mappings, the target token ID is used directly from the mapping entry.
14. **Provenance Preservation:** Emit the translated token as a structured package containing: the target token representation; the source semantic token's provenance chain, embedded as a read-only provenance artifact; and the mapping metadata (type, confidence, authority, validity period). The receiving system is expected to preserve this embedded provenance artifact and make it queryable through its own provenance query interface.
15. **Provenance Recording:** Append an `INTEROP_MAPPING` provenance event to the source token's provenance chain, recording the target system, mapping type, translation timestamp, and the identity of the STP authority.

## 11.5 Interoperability with External Ontologies

A significant category of interoperability mappings involves alignment between the organization's internal semantic fields and externally defined, industry-standard ontologies. Such ontologies provide a shared vocabulary that enables semantic interoperability not just between internal systems but across industry boundaries. The framework provides structured support for mapping to and from the following industry-standard ontologies:

| Ontology                                              | Domain                 | Primary Use Case                                                                    |
|-------------------------------------------------------|------------------------|-------------------------------------------------------------------------------------|
| SKOS (Simple Knowledge Organization System)           | General / Cross-domain | Vocabulary alignment, concept mapping, thesaurus management                         |
| OWL (Web Ontology Language)                           | General / Cross-domain | Formal ontology alignment, class equivalence, property mapping                      |
| Schema.org                                            | Web / General Business | Structured data interoperability, SEO, open data publishing                         |
| FIBO (Financial Industry Business Ontology)           | Financial Services     | Financial instrument classification, legal entity identification, contract modeling |
| HL7 FHIR (Fast Healthcare Interoperability Resources) | Healthcare             | Clinical data exchange, patient identity, clinical concept mapping                  |
| SNOMED CT                                             | Healthcare / Clinical  | Clinical terminology normalization, diagnosis and procedure coding                  |
| GS1 (Global Standard 1)                               | Supply Chain / Retail  | Product identification, GTIN mapping, supply chain data exchange                    |
| LEI (Legal Entity Identifier) / GLEIF                 | Financial / Regulatory | Legal entity identification, regulatory reporting, counterparty identification      |

Ontology alignment is a two-phase process. In the first phase, an automated alignment tool identifies candidate concept mappings between the internal field ontology and the external ontology, using a combination of label similarity, structural graph analysis, and existing cross-ontology alignments. The automated phase produces a candidate mapping set with confidence scores. In the second phase, a human ontology expert reviews the candidate mappings, confirms correct mappings, rejects incorrect ones, and creates manual mappings for concepts that the automated tool could not align. The confirmed mapping set is registered in the Interoperability Mapping Registry and is applied to all tokens bound to the relevant semantic field. Ontology version mismatches — arising when the external ontology publishes a new version — are handled by re-running the automated alignment phase against the new version and submitting the delta mappings for governance review.

## 11.6 Cross-Organization Semantic Contracts

When two or more organizations wish to establish a formal, governed basis for semantic token exchange, they execute a **Semantic Contract**: a bilateral or multilateral agreement that specifies the semantic fields covered, the interoperability mappings agreed upon, the governing ontologies recognized by each party, and the dispute resolution mechanism to be applied when the parties disagree on a mapping or equivalence assertion.

The Semantic Contract schema includes the following elements:

- **contract\_id**: A globally unique identifier for the contract, issued by a neutral registry or agreed upon by the parties.
- **parties**: The organization identifiers (e.g., LEI codes) of all parties to the contract.
- **effective\_date and expiry\_date**: The validity period of the contract.
- **covered\_fields**: The semantic fields within each party's semantic token store that are covered by the contract, with the field version applicable at contract execution.
- **mapping\_registry\_reference**: A reference to the shared mapping registry (or a copy thereof) that specifies the agreed interoperability mappings between the parties' respective token spaces.
- **dispute\_resolution\_mechanism**: The agreed procedure for resolving equivalence or mapping disputes, including the escalation path, resolution timeline, and binding arbitration mechanism.
- **amendment\_protocol**: The procedure by which the contract may be amended as field definitions, ontologies, or mapping requirements evolve.

Semantic contracts are versioned and subject to the semantic token lifecycle model: a newly executed contract is in ACTIVE state; a superseded contract (replaced by a new version) is in DEPRECATED state; an expired or terminated contract is in RETIRED state. Tokens exchanged under a semantic contract carry the contract\_id as a reference in their INTEROP\_MAPPING provenance events, ensuring that the full contractual basis for every cross-organization token exchange is fully traceable.

## 11.7 Federation and Distributed Semantic Stores

In federated deployment architectures — common in large enterprises, industry consortia, and multi-organization data ecosystems — each participating organization maintains its own semantic token store, governed by its own Semantic Governance Board and field stewardship structure. A **Federation Layer** provides cross-organization equivalence resolution and interoperability translation without requiring any organization to surrender governance of its own semantic token store.

The Federation Layer maintains a **Federation Registry** — a shared catalog of all participant organizations, their semantic field registries (at a summary level), their semantic contracts, and the federation-level equivalence classes that span organizational boundaries. When a consuming system in Organization A requests the resolution of a token that originated in Organization B, the Federation Layer: (a) validates that Organization A holds a valid semantic contract with Organization B covering the relevant field; (b) retrieves the source token from Organization B's semantic store (or from a federation cache, if available); (c) applies the STP using the contract's mapping registry; and (d) returns the translated token to Organization A's consuming system.

Consistency guarantees in federated deployments are eventual rather than strong: because each organization's semantic token store is independently governed, updates to one organization's token definitions or field versions may not be immediately reflected in the federation registry. The Federation Layer publishes a defined propagation latency bound (default: 24 hours) within which federation registry updates are guaranteed to be consistent with the source organization's state. Consuming systems in federated deployments must be designed to tolerate this propagation latency and must not assume strong consistency guarantees for cross-organization token resolution.

## 12. Reductions to Practice

---

### 12.1 Implementation Overview

This section describes four concrete implementation scenarios that demonstrate how the semantic tokenization framework addresses specific, high-value enterprise problems. Each reduction to practice is structured to describe the problem context, the specific framework components that are applied, and the measurable outcomes that result. These scenarios are drawn from representative enterprise architecture patterns and are intended to illustrate the practical value of the framework rather than to describe specific deployments.

### 12.2 Reduction 1: Enterprise Customer Identity Resolution

**Problem Context.** A large enterprise operates a fragmented customer data landscape in which the same customer may be represented by different identifiers in its CRM system, ERP system, billing platform, and customer support system. Each system has developed its own customer identification convention, and the same customer may appear as "CUST-00124891" in the CRM, "BIL-2019-87432" in the billing platform, and "SUPP-443-EUR" in the support system. No authoritative cross-system customer identity exists; reconciliation is performed manually, episodically, and incompletely. The consequences include fragmented customer analytics, inconsistent customer communications, GDPR compliance gaps due to inability to honor data subject access requests across all systems, and inability to produce a unified customer view for regulatory reporting purposes.

**Framework Application.** The enterprise applies the canonical tokenization layer (per the companion paper) to normalize the structural form of all customer identifiers across its four systems, producing a set of canonical customer tokens. It then applies the semantic tokenization framework as follows: (1) A "Customer Identity" semantic field is registered in the Semantic Field Registry, governed by the enterprise's proprietary customer ontology and version-controlled; (2) all canonical customer tokens are bound to the "Customer Identity" field via the field binding mechanism, using the source system identifier as the primary context signal; (3) the normalization pipeline resolves each bound token to its SNF — the enterprise's authoritative customer concept URI, which includes the resolved customer ID, name, contact information binding, and applicable regulatory classification; (4) the equivalence evaluator places all canonical customer tokens that resolve to the same SNF into a cross-system equivalence class, establishing the formal "same customer" relationship; (5) compliance tags are applied for GDPR Article 5 (accuracy), GDPR Article 17 (erasure), and the enterprise's internal customer data privacy policy; and (6)

triggers are defined to notify the data governance team when any customer token transitions to DEPRECATED or QUARANTINED state, initiating the appropriate downstream reconciliation workflow.

**Outcome.** Any system that receives a semantic customer token can resolve it to the authoritative SNF, retrieve all equivalent tokens in other systems via the equivalence class, determine the token's current lifecycle state, and check compliance with the enterprise's data privacy obligations — all from a single, governed, provenance-backed source of semantic truth. The GDPR data subject access request process, previously a multi-week manual effort, is reduced to an automated query against the semantic token store that retrieves all tokens in the requesting customer's equivalence class and produces a compliance-tagged record of all data held across all systems.

### 12.3 Reduction 2: Regulatory Reporting Semantic Compliance

**Problem Context.** A financial institution is required to submit quarterly regulatory reports to a central bank, with all data elements expressed using the regulator's mandated reporting ontology. The institution's source data systems use heterogeneous internal terminologies that have evolved over decades; a single reported data element (e.g., "counterparty legal entity type") may be expressed using any of fourteen different internal codes across the institution's systems, only some of which map cleanly to the regulator's mandated classification scheme. Manual mapping is performed by a dedicated regulatory reporting team, but the process is slow, error-prone, and produces inconsistent results across reporting periods. Regulatory examiners have questioned the accuracy of several submissions, and the institution faces the prospect of material misstatement findings.

**Framework Application.** The institution applies semantic tokenization to all data elements subject to regulatory reporting requirements: (1) A "Regulatory Reporting" semantic field is registered, governed by the regulator's published ontology (loaded as the field's governing ontology at its published version); (2) all source data tokens for reportable data elements are bound to the "Regulatory Reporting" field, with normalization rules that map each of the fourteen internal codes to the correct regulator ontology concept URI; (3) compliance tags for the applicable reporting framework (e.g., a custom framework modeled on the central bank's reporting instructions) are applied to every token at normalization time; (4) a point-in-time compliance snapshot is generated for each reporting period and included as the compliance evidence artifact in the regulatory submission package; (5) audit-ready artifacts — including the full provenance chain for every reported token, cryptographically signed — are archived and made available to examiners on demand.



**Outcome.** The institution produces fully traceable, audit-ready regulatory submissions in which every data element carries machine-verifiable evidence of its correct semantic interpretation, the rules applied to produce that interpretation, the authority that approved it, and the compliance evaluation that confirmed its regulatory fitness. Examiner challenges are resolved by presenting the tamper-evident provenance artifact for the challenged element, demonstrating precisely how the reported value was derived. Regulatory misstatement risk is materially reduced, and the regulatory reporting team is redeployed from manual mapping to higher-value governance activities.

#### **12.4 Reduction 3: Cross-Organization Data Exchange**

**Problem Context.** Two partner organizations — a pharmaceutical manufacturer and a wholesale distributor — exchange product data on a daily basis as part of their supply chain integration. The manufacturer uses a proprietary product taxonomy with approximately 12,000 product concepts; the distributor uses the GS1 product classification scheme. The two taxonomies overlap substantially but not completely: approximately 85% of the manufacturer's products map cleanly to GS1 classes, 10% require transformation logic, and 5% have no GS1 equivalent. Currently, cross-organization product mapping is maintained in a shared spreadsheet that is updated manually and is known to be out of date. Mapping errors cause fulfillment failures, incorrect pricing, and regulatory non-compliance for certain controlled product categories.

**Framework Application.** Each organization applies semantic tokenization to its product tokens within its own governance domain. They then execute a semantic contract that: (1) designates the manufacturer's "Product Classification" field and the distributor's "GS1 Product" field as the covered fields; (2) registers a formal mapping registry (replacing the shared spreadsheet) with EXACT mappings for the 85%, RELATED mappings (with documented transformation logic) for the 10%, and a flag for human review for the 5%; (3) establishes a bilateral authority for maintaining and updating the mapping registry; and (4) specifies a dispute resolution procedure for cases in which the organizations disagree on a mapping. The Semantic Translation Protocol is implemented at the API boundary between the two organizations; on each product data exchange, the STP translates the source token to the target taxonomy, preserves the source provenance chain as an embedded artifact, and records an INTEROP\_MAPPING provenance event.

**Outcome.** The receiving organization can consume the partner's product data with full semantic fidelity: it knows precisely which product was intended, what mapping type was applied, what transformation logic was used, and what confidence level applies to the mapping. Fulfillment failures attributable to product identity errors are eliminated for the 95% of products with EXACT or RELATED mappings. The 5% requiring human review are surfaced automatically via trigger actions rather than discovered through fulfillment failures. The mapping registry, now a governed artifact with lifecycle management, is updated through a formal amendment protocol rather than informal spreadsheet edits.

## 12.5 Reduction 4: AI/ML Training Data Governance

**Problem Context.** A technology organization develops machine learning models trained on large datasets containing concept labels — categories, entities, intents, and classifications — that have been accumulated over several years from multiple internal and external sources. Over time, the meaning of many concept labels has shifted: the category "Sensitive Customer Data" was redefined twice in the past three years as privacy regulations evolved; the entity label "Premium Customer" was reclassified to include a different customer tier after a business model change; and several intent labels used in a conversational AI training set were found to encode demographic biases and were subsequently retired. Models trained on datasets that predate these changes exhibit unexpected behavior — what is now called semantic drift — because they learned associations based on meanings that no longer apply. Retraining is expensive, and the organization cannot reliably determine which trained models are affected by which meaning changes.

**Framework Application.** The organization applies semantic tokenization to all concept labels in its training datasets: (1) A "ML Training Concepts" semantic field is registered for each model domain (e.g., "Customer Classification," "Intent Recognition," "Sentiment Analysis"), governed by a domain-specific ontology that the organization's ML governance team maintains; (2) at dataset creation time, all concept labels are bound to their respective semantic fields, and the SNF at that field version is recorded in the dataset metadata; (3) the lifecycle state of each concept label token is recorded at the time the dataset is assembled; (4) triggers are defined such that when any concept label token's SNF changes (due to a field version update) or its lifecycle state transitions to DEPRECATED or RETIRED, a QUARANTINE action is initiated for all training datasets that contain that token in an ACTIVE state, and a NOTIFY action is sent to the affected model owners; (5) compliance tags for internal ML governance policies (covering bias assessment, privacy protection, and concept currency requirements) are applied and maintained through the standard compliance evaluation workflow.

**Outcome.** The organization can, for the first time, produce an authoritative, machine-verifiable answer to the question: "What did concept label X mean at the time dataset D was assembled, and is that meaning still current?" Models trained on datasets with semantically governed labels are auditable in a way that was previously impossible; the training data's semantic state at training time is preserved in the provenance chain and can be reconstructed at any future point. When a concept label's meaning changes, all affected training datasets are automatically quarantined and their owners notified, preventing the continued production of models trained on semantically stale data without explicit governance acknowledgment. The cost of reactive model retraining due to undiscovered semantic drift is materially reduced.

## 13. Discussion

---

### 13.1 Relationship to Canonical Tokenization

The relationship between semantic tokenization and canonical tokenization is one of architectural layering with mutual dependency. Semantic tokenization is architecturally positioned above canonical tokenization and depends on it as a mandatory prerequisite: no semantic token can be produced without a canonical token as its base. Conversely, canonical tokenization alone does not provide a complete token governance solution; it resolves form but not meaning, and it is the resolution of meaning that enables the governance, compliance, and interoperability capabilities that enterprise data architectures require.

Together, the two frameworks constitute what this paper terms the **Complete Token Governance Stack**: a layered architecture in which every token is governed at the level of form (canonical tokenization) and at the level of meaning (semantic tokenization), with every transition between levels producing a richer, more governable artifact. Organizations that have already adopted canonical tokenization per the companion paper have a clear and well-defined integration path to semantic tokenization; the canonical token is the entry point, and the semantic enrichment pipeline described in Section 3.4 provides the bridge. Organizations that have not yet adopted canonical tokenization should treat it as the first phase of a two-phase token governance initiative, with semantic tokenization as the second.

### 13.2 Scalability and Performance Considerations

Semantic resolution is more computationally intensive than canonical resolution due to the additional processing steps — ontology traversal, equivalence class evaluation, trigger condition evaluation, compliance checking — that operate above the canonical layer. In high-throughput environments (e.g., real-time financial transaction streams, large-scale healthcare data ingestion), the latency introduced by semantic resolution must be carefully managed to avoid creating a governance bottleneck.

The primary scalability mechanism is caching. The Semantic Resolution Engine maintains a multilevel cache: a *SNF Cache* keyed on (canonical\_token\_id, field\_id, field\_version) that stores pre-computed SNFs for recently resolved tokens; an *Equivalence Class Cache* that stores pre-computed equivalence class memberships for recently evaluated token pairs; and a *Trigger Evaluation Cache* that stores the set of applicable triggers for each (field\_id, lifecycle\_state) combination, reducing the cost of trigger lookup on each evaluation. Cache invalidation is event-driven: when a field version is updated, the SNF Cache entries for all tokens bound to that field are invalidated; when an equivalence class is updated, the relevant Equivalence Class Cache entries are invalidated.

Horizontal scaling of the SRE is supported through a stateless processing model: all state (token records, field definitions, equivalence classes, trigger registries, compliance tags) is maintained in distributed storage systems external to the SRE processing nodes. SRE nodes are stateless and can be scaled horizontally to handle increased token resolution throughput without any changes to the storage layer. The caching layer is implemented as a distributed cache (e.g., Redis Cluster or equivalent) shared across all SRE nodes, ensuring that cache hits for one node benefit all nodes. For environments requiring extremely low latency, a subset of frequently resolved tokens may be designated for local in-memory caching at the SRE node level, with a defined staleness bound.

### 13.3 Governance and Organizational Change Management

The successful deployment of the semantic tokenization framework requires not only technical implementation but significant organizational investment in semantic governance capabilities. The framework introduces several governance roles and processes that are new to most organizations: the Semantic Governance Board, Field Stewards, the Central Ontology Team, and the semantic analyst role. Establishing and sustaining these roles requires executive sponsorship, clear role definition, training, and integration with existing data governance structures.

The recommended governance structure comprises three tiers. The **Semantic Governance Board** is the apex governance body, responsible for approving new semantic fields, resolving equivalence disputes,

adjudicating compliance violations, and setting organization-wide semantic governance policies. It should include representation from legal/compliance, data architecture, domain leadership, and external regulatory advisors. **Field Stewards** are domain experts assigned to each registered semantic field; they own the field definition, approve token bindings and normalization rules, manage field versioning, and serve as the first point of escalation for field-specific issues. The **Central Ontology Team** maintains the organization's internal ontologies, manages alignment with external ontologies, and provides technical support to Field Stewards on ontology-related questions.

### 13.4 Limitations and Open Problems

Three substantive limitations of the current framework merit explicit acknowledgment. The first is the *bootstrapping problem*: before the semantic tokenization framework can operate, a substantial initial investment is required to define semantic fields, populate their governing ontologies, establish normalization rules, and onboard the first wave of tokens. This investment is front-loaded and may be difficult to justify without clear demonstration of return. Organizations are advised to begin with a single high-value semantic field — typically the field for which meaning ambiguity has the most costly known consequences — and to expand from that beachhead rather than attempting a full-organization rollout simultaneously.

The second limitation is *semantic drift*: the gradual shift in the meaning of concepts over time. The lifecycle management and field versioning mechanisms described in this paper provide tools for managing known meaning changes — changes that are identified by field stewards and captured through formal field version updates. They do not, however, provide mechanisms for detecting unknown or unacknowledged meaning changes: cases in which the real-world referent of a concept shifts without the change being captured in the governing ontology. This is an open problem in knowledge management generally, and the framework's current provisions address it only through the mandatory review policies (Section 7.4), which ensure that field definitions are periodically revisited.

The third limitation is *equivalence completeness*: no equivalence class registry can be exhaustive. There will always be pairs of tokens whose equivalence relationship has not yet been explicitly established in the registry, and the approximate equivalence mechanisms — ontology traversal and similarity scoring — cannot guarantee correct results for all such pairs. This incompleteness is bounded and disclosed: the confidence scores on approximate equivalence results communicate the degree of uncertainty, and the resolution path recorded in the EQUIVALENCE\_RESOLVED provenance event makes the basis of the equivalence determination explicit. Consuming systems must treat approximate equivalence results with

appropriate caution and must not use them as a basis for compliance-critical decisions without additional human validation.

## 14. Conclusion

---

This paper has presented Semantic Tokenization as a comprehensive, formally specified framework for the assignment, management, governance, and exchange of meaning-bearing identifiers in enterprise data architectures. The framework is organized around eight core components — Semantic Fields, Semantic Normalization, Semantic Provenance, Semantic Lifecycle, Semantic Equivalence, Semantic Triggers, Semantic Compliance, and Semantic Interoperability — each of which addresses a distinct dimension of the problem of meaning governance in token-based systems. Together, these components constitute a complete solution to the semantic gap that canonical tokenization alone cannot bridge.

The complementary relationship between semantic tokenization and canonical tokenization is fundamental to both frameworks' value proposition. Canonical tokenization resolves the question of form: it ensures that the same entity is always represented by the same structural identifier. Semantic tokenization resolves the question of meaning: it ensures that the same entity's meaning — what it refers to, in what domain, under what ontological authority, at what point in time — is always explicitly encoded, versioned, and governed. Together, the two frameworks provide the minimum necessary condition for a token-based data architecture that is simultaneously syntactically consistent and semantically trustworthy.

The four reductions to practice presented in Section 12 demonstrate that the framework's benefits are not merely theoretical. Enterprise customer identity resolution, regulatory reporting compliance, cross-organization data exchange, and AI/ML training data governance are all domains in which meaning ambiguity imposes concrete, measurable costs — costs that the semantic tokenization framework is designed to eliminate through architectural discipline rather than manual effort. The framework's trigger-driven automation, provenance-backed auditability, and lifecycle-governed token management transform semantic governance from a periodic, manual activity into a continuous, systematic, and scalable capability.

This paper proposes that semantic tokenization be adopted as an enterprise-grade standard for meaning-preserving identity resolution across all industries in which token-based data exchange carries regulatory, financial, or operational significance. The framework's formal specification — including the semantic

token schema, the Semantic Normal Form definition, the provenance chain structure, the lifecycle state model, the equivalence resolution algorithm, and the Semantic Translation Protocol — provides sufficient detail for implementation teams to build conformant systems, for governance teams to establish appropriate policies, and for auditors to verify compliance.

Future directions for the framework include three areas of active research and development. First, *automated ontology learning*: the application of large language models and knowledge graph embedding techniques to automate the discovery of semantic fields, the population of governing ontologies, and the identification of normalization rules from existing organizational data — reducing the bootstrapping investment required for new field definitions. Second, *federated semantic governance*: the extension of the federation architecture to support cross-industry semantic contracts, enabling meaning-preserving data exchange between organizations in different industries that share overlapping concepts but use different domain ontologies. Third, *AI-assisted equivalence discovery*: the deployment of machine learning models trained on existing equivalence class registries to proactively identify candidate equivalence relationships for equivalence classes that have not yet been explicitly established — transforming equivalence discovery from a reactive, event-driven process into a proactive, continuously improving capability.

The authors invite organizations implementing this framework to contribute to its ongoing development through the formal amendment process described in the companion paper, ensuring that the framework evolves in response to real-world deployment experience while maintaining the formal rigor and governance discipline that are its defining characteristics.

The following terms are defined as used in this paper. All definitions are authoritative within the scope of the Semantic Tokenization framework. Where a term is also used in the companion Canonical Tokenization paper, the definition given here is consistent with and extends the companion paper's definition.

| Term                                    | Definition                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Canonical Token (CT)</b>             | A token that has been structurally normalized to a single authoritative form by the canonical tokenization layer, eliminating representational variation. The canonical token is the mandatory input to the semantic tokenization layer. Defined in detail in the companion paper, <i>Canonical Tokenization: A Framework for Structural Identity Normalization</i> .                                 |
| <b>Complete Token Governance Stack</b>  | The four-layer architecture combining canonical tokenization (Layer 1) and semantic tokenization (Layers 2–4) to provide comprehensive governance of token form and meaning. A token that has been processed through the Complete Token Governance Stack is structurally normalized, semantically resolved, equivalence-evaluated, trigger-processed, compliance-tagged, and interoperability-mapped. |
| <b>Compliance Tag</b>                   | A structured metadata attribute of a semantic token that records the result of evaluating the token against a specific requirement of a specific compliance framework. Each compliance tag carries a status (SATISFIED, PENDING, VIOLATED, or EXEMPT), the assessment timestamp, the assessing authority, and an evidence reference. See Section 10.2.                                                |
| <b>Equivalence Class</b>                | A formally registered set of semantic tokens that are interchangeable within a defined semantic field and context, as determined by the Equivalence Evaluator and recorded in the Equivalence Class Registry. Equivalence classes are field-scoped, version-scoped, and confidence-bounded. See Section 8.2.                                                                                          |
| <b>Field Steward</b>                    | A domain expert assigned governance responsibility for a specific registered semantic field. The Field Steward approves token bindings, manages field versioning, approves normalization rules, and is the first point of escalation for field-specific semantic issues. See Section 13.3.                                                                                                            |
| <b>Interoperability Mapping</b>         | A formally specified relationship between an internal semantic token and the corresponding token representation in an external system or ontology. Interoperability mappings carry a mapping type (EXACT, BROADER, NARROWER, RELATED, or CUSTOM), a confidence score, and a validity period. See Section 11.2.                                                                                        |
| <b>Normalization Rules Engine (NRE)</b> | The component of the Semantic Resolution Engine responsible for applying field-specific normalization rules — including synonym rules, abbreviation expansion rules, domain-specific alias rules, and cross-lingual equivalence rules — to produce a candidate concept URI for semantic normal form resolution. See Section 5.4.                                                                      |



| Term                              | Definition                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Provenance Chain</b>           | An ordered, append-only log of provenance events associated with a semantic token, recording the complete lineage of the token's meaning from creation to the present. Provenance events are cryptographically signed by their acting authorities and may never be modified or deleted. See Section 6.2.                                     |
| <b>Semantic Compliance</b>        | The property of a semantic token whereby its meaning, field binding, provenance chain, and lifecycle state jointly satisfy all applicable governance policies and regulatory obligations, as evaluated by the Compliance Checker and recorded in the token's compliance tags. See Section 10.                                                |
| <b>Semantic Contract</b>          | A formal bilateral or multilateral agreement between two or more organizations that establishes the governing basis for semantic token exchange between them, including the covered semantic fields, the agreed interoperability mappings, the recognized governing ontologies, and the dispute resolution mechanism. See Section 11.6.      |
| <b>Semantic Equivalence</b>       | The relationship between two or more semantic tokens that refer to the same underlying concept within a given semantic field and field version, as determined by the Equivalence Resolution Algorithm. Semantic equivalence is field-scoped, version-scoped, and confidence-bounded. See Section 8.1.                                        |
| <b>Semantic Field</b>             | A bounded, formally registered domain of meaning within which a token's interpretation is specified, governed, and versioned. Semantic fields act as meaning namespaces: two tokens are comparable in meaning only if they share the same semantic field and field version. See Section 4.1.                                                 |
| <b>Semantic Gap</b>               | The divergence between syntactic equivalence and semantic equivalence: the condition in which two tokens that are canonically identical (same form) carry different meanings in different contexts, domains, or time periods. The semantic gap is the foundational problem that semantic tokenization is designed to resolve.                |
| <b>Semantic Governance Board</b>  | The apex governance body responsible for approving new semantic fields, resolving equivalence disputes, adjudicating compliance violations, managing the authority registry, and setting organization-wide semantic governance policies. See Section 13.3.                                                                                   |
| <b>Semantic Interoperability</b>  | The capability to exchange semantic tokens across system and organizational boundaries while preserving the meaning that the source system intended to convey. Semantic interoperability is enabled by the Interoperability Mapping Architecture, the Semantic Translation Protocol, and Semantic Contracts. See Section 11.                 |
| <b>Semantic Lifecycle</b>         | The defined set of operational states (DRAFT, PENDING_REVIEW, ACTIVE, DEPRECATED, RETIRED, QUARANTINED, CONTESTED) through which every semantic token may transition, governed by a defined state machine, transition guards, and lifecycle policies. The lifecycle state is a first-class attribute of every semantic token. See Section 7. |
| <b>Semantic Normal Form (SNF)</b> | The unique, authoritative representation of a concept's meaning within a given semantic field and field version, comprising the resolved concept URI, preferred label, field-specific attributes, disambiguation qualifiers, SNF version, and                                                                                                |

| Term                                       | Definition                                                                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                            | normalization confidence score. The SNF is unique per (canonical_token_id, semantic_field_id, field_version) triple. See Section 5.2.                                                                                                                                                                                                                                                                                                   |
| <b>Semantic Normalization</b>              | The process of resolving interpretive variation in token meaning: given any valid expression of a concept within a defined semantic field, producing a single, unambiguous Semantic Normal Form. Semantic normalization operates above canonical normalization (which resolves structural variation). See Section 5.                                                                                                                    |
| <b>Semantic Provenance</b>                 | The complete, tamper-evident record of a semantic token's meaning lineage: every decision, transformation, assignment, and validation that has contributed to the token's current semantic state. Semantic provenance is the audit trail of meaning. See Section 6.                                                                                                                                                                     |
| <b>Semantic Resolution Engine (SRE)</b>    | The central processing component of the semantic tokenization framework, responsible for accepting canonical tokens and resolution contexts as input and producing fully populated semantic tokens as output. The SRE coordinates seven internal components: Field Classifier, Normal Form Generator, Provenance Recorder, Equivalence Evaluator, Trigger Dispatcher, Compliance Checker, and Interoperability Mapper. See Section 3.3. |
| <b>Semantic Token (ST)</b>                 | A token that carries not only a canonical identifier but also a bounded, versioned, and governed meaning context, formally defined as the five-tuple $ST = (CT, SF, SN, SP, SL)$ : canonical token, semantic field, semantic normal form, semantic provenance chain, and semantic lifecycle state. See Section 2.1.                                                                                                                     |
| <b>Semantic Token Schema</b>               | The defined structure of the semantic token record, specifying all fields, their types, cardinality constraints, and purposes. The schema is versioned and governed; core schema fields are immutable within a schema version. See Section 3.2.                                                                                                                                                                                         |
| <b>Semantic Translation Protocol (STP)</b> | The defined procedure by which a semantic token is translated when it crosses a system or organizational boundary, ensuring that the receiving system receives not only a transformed token identifier but the full semantic context necessary to interpret it correctly, including the source SNF and provenance chain. See Section 11.4.                                                                                              |
| <b>Semantic Trigger</b>                    | A declarative, governed rule that specifies a condition over token semantic state and context, and an action to be executed when that condition evaluates as true. Semantic triggers are governed artifacts with their own lifecycle and are the mechanism by which event-driven semantic governance is implemented without embedding semantic logic in consuming applications. See Section 9.                                          |
| <b>Trigger Condition Language (TCL)</b>    | A domain-specific declarative expression language for defining predicates over semantic token state, evaluated by the Trigger Evaluation Engine. TCL is intentionally non-Turing-complete to enable static analysis and efficient evaluation. See Section 9.3.                                                                                                                                                                          |
| <b>Trigger Evaluation Engine (TEE)</b>     | The event-driven subsystem of the SRE that receives token state change events and evaluates all in-scope, ACTIVE triggers against each event, dispatching applicable trigger actions synchronously or asynchronously per their defined execution mode. See Section 9.2.                                                                                                                                                                 |

---

Semantic Tokenization: A Framework for Meaning-Preserving Identity Resolution | Proprietary & Confidential | Version 1.0 | July 2025

A Companion Paper to *Canonical Tokenization: A Framework for Structural Identity Normalization* | Token Governance Architecture Series, Volume II

Unauthorized reproduction or distribution is strictly prohibited. All rights reserved.



## FURTHER READING AND REFERENCES

---

### FOUNDATIONAL WORKS IN TOKENIZATION AND IDENTITY

- [1] Agrawal, R., & Srikant, R. (1994). Fast Algorithms for Mining Association Rules in Large Databases. Proceedings of the 20th International Conference on Very Large Data Bases (VLDB), 487–499.
- [2] Bertino, E., & Sandhu, R. (2005). Database Security — Concepts, Approaches, and Challenges. IEEE Transactions on Dependable and Secure Computing, 2(1), 2–19.
- [3] PCI Security Standards Council. (2011). PCI DSS Tokenization Guidelines. PCI SSC Information Supplement. Wakefield, MA: PCI Security Standards Council.
- [4] NIST. (2017). Digital Identity Guidelines (SP 800-63-3). National Institute of Standards and Technology, U.S. Department of Commerce.  
  
<https://doi.org/10.6028/NIST.SP.800-63-3>

- [5] Gruber, T. R. (1993). A Translation Approach to Portable Ontology Specifications. *Knowledge Acquisition*, 5(2), 199–220.
- [6] Guarino, N. (1998). Formal Ontology in Information Systems. *Proceedings of the First International Conference on Formal Ontology in Information Systems (FOIS '98)*, IOS Press, 3–15.
- [7] Noy, N. F., & McGuinness, D. L. (2001). *Ontology Development 101: A Guide to Creating Your First Ontology*. Stanford Knowledge Systems Laboratory Technical Report KSL-01-05.
- [8] W3C. (2009). SKOS Simple Knowledge Organization System Reference. W3C Recommendation. World Wide Web Consortium.  
<https://www.w3.org/TR/skos-reference/>
- [9] Bechhofer, S., van Harmelen, F., Hendler, J., Horrocks, I., McGuinness, D. L., Patel-Schneider, P. F., & Stein, L. A. (2004). *OWL Web Ontology Language Reference*. W3C Recommendation. World Wide Web Consortium.  
<https://www.w3.org/TR/owl-ref/>
- [10] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34–43.

SEMANTIC NORMALIZATION AND EQUIVALENCE[11] Codd, E. F. (1972). Further Normalization of the Data Base Relational

Model. In R. Rustin (Ed.), Data Base Systems: Courant Computer Science

Symposia Series, Vol. 6. Englewood Cliffs, NJ: Prentice-Hall.

[12] Euzenat, J., & Shvaiko, P. (2013). Ontology Matching (2nd ed.).

Heidelberg: Springer.

[13] Levenshtein, V. I. (1966). Binary Codes Capable of Correcting Deletions,

Insertions, and Reversals. Soviet Physics Doklady, 10(8), 707–710.

[14] Doan, A., Madhavan, J., Domingos, P., & Halevy, A. (2002). Learning to

Map Between Ontologies on the Semantic Web. Proceedings of the 11th

International World Wide Web Conference (WWW2002), 662–673.

- [15] Buneman, P., Khanna, S., & Tan, W. C. (2001). Why and Where: A Characterization of Data Provenance. Proceedings of the 8th International Conference on Database Theory (ICDT 2001), Lecture Notes in Computer Science, Vol. 1973, 316–330.
- [16] Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., & Van den Bussche, J. (2011). The Open Provenance Model — Core Specification (v1.1). Future Generation Computer Systems, 27(6), 743–756.
- [17] W3C. (2013). PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. <https://www.w3.org/TR/prov-dm/>
- [18] Cheney, J., Chiticariu, L., & Tan, W. C. (2009). Provenance in Databases: Why, How, and Where. Foundations and Trends in Databases, 1(4), 379–474.



- [19] DAMA International. (2017). DAMA-DMBOK: Data Management Body of Knowledge (2nd ed.). Technics Publications.
- [20] ISO/IEC 38505-1:2017. (2017). Information Technology — Governance of IT — Governance of Data — Part 1: Application of ISO/IEC 38500 to the Governance of Data. Geneva: International Organization for Standardization.
- [21] Loshin, D. (2010). Master Data Management. Burlington, MA: Morgan Kaufmann.
- [22] Seiner, R. S. (2014). Non-Invasive Data Governance: The Path of Least Resistance and Greatest Success. Technics Publications.

## SEMANTIC COMPLIANCE AND REGULATORY FRAMEWORKS

- [23] European Parliament. (2016). Regulation (EU) 2016/679 of the European Parliament and of the Council (General Data Protection Regulation).  
Official Journal of the European Union, L 119, 1–88.
- [24] U.S. Department of Health and Human Services. (2002). Standards for Privacy of Individually Identifiable Health Information (HIPAA Privacy Rule). 45 C.F.R. Parts 160 and 164.
- [25] NIST. (2018). Framework for Improving Critical Infrastructure Cybersecurity, Version 1.1. National Institute of Standards and Technology.  
<https://doi.org/10.6028/NIST.CSWP.04162018>
- [26] Gellert, R. (2020). The Risk-Based Approach to Data Protection. Oxford: Oxford University Press.

- [27] Sheth, A. P., & Larson, J. A. (1990). Federated Database Systems for Managing Distributed, Heterogeneous, and Autonomous Databases. *ACM Computing Surveys*, 22(3), 183–236.
- [28] Hull, R. (1997). Managing Semantic Heterogeneity in Databases: A Theoretical Perspective. *Proceedings of the 16th ACM Symposium on Principles of Database Systems (PODS '97)*, 51–61.
- [29] Lenzerini, M. (2002). Data Integration: A Theoretical Perspective. *Proceedings of the 21st ACM Symposium on Principles of Database Systems (PODS '02)*, 233–246.
- [30] Bernstein, P. A., Halevy, A. Y., & Pottinger, R. A. (2000). A Vision for Management of Complex Models. *ACM SIGMOD Record*, 29(4), 55–63.

## INDUSTRY STANDARDS AND REFERENCE ARCHITECTURES

[31] Object Management Group (OMG). (2016). Financial Industry Business

Ontology (FIBO). EDM Council and OMG Joint Specification.

<https://spec.edmcouncil.org/fibo/>

[32] HL7 International. (2019). HL7 FHIR Release 4: Fast Healthcare

Interoperability Resources. HL7 International.

<https://hl7.org/fhir/R4/>

[33] GS1. (2022). GS1 General Specifications, Version 22. GS1 Global Office,

Brussels, Belgium. [https://www.gs1.org/docs/barcodes/GS1\\_General\\_](https://www.gs1.org/docs/barcodes/GS1_General_Specifications.pdf)

[Specifications.pdf](https://www.gs1.org/docs/barcodes/GS1_General_Specifications.pdf)

[34] The Open Group. (2018). The TOGAF Standard, Version 9.2. The Open Group

Series. Reading, UK: The Open Group.

[35] schema.org Community Group. (2015). Schema.org: A Shared Vocabulary for

Structured Data. <https://schema.org/>

## COMPANION WORK

[36] Leon Calvin Long II (2025). Canonical Tokenization: A New Architecture for Trustless

Document Permanence. [Proprietary Technical White Paper, Version 1.0].